Tugas Besar – IF3070 Dasar Intelegensi Artifisial
Implementasi Algoritma Local Search dalam Penyelesaian Masalah pada Studi Kasus



Oleh Kelompok UnaYahaPrrri:

Endda Tsa Azzahra Syaifur	18224094
Rafael Sean Hadipranoto	18224100
Gifta Mooira Prasharnissa	18224106

Program Studi Sistem dan Teknologi Informasi
Institut Teknologi Bandung – 2026

Daftar Isi

Daftar Isi **2**
Bab I: Latar Belakang **3**
1.1 Latar Belakang 3
Bab II: Pembahasan **4**
2.1 Penentuan Variabel dan Spesifikasi 4
2.2 Penyelesaian dengan Hill Climbing 5
2.2.1 Implementasi Algoritma 5
2.2.2 Hasil Algoritma 14
2.3 Penyelesaian dengan Simulated Annealing 19
2.3.1 Implementasi Algoritma 21
2.3.2 Hasil Algoritma 29
2.4 Penyelesaian dengan Genetic Algorithm 32
2.4.1 Implementasi Algoritma 34
2.4.2 Hasil Algoritma 43
Bab III: Kesimpulan dan Saran **63**
3.1 Kesimpulan 63
3.2 Saran 64
Pembagian Tugas **65**
Referensi **66**
Lampiran **67**
Surat Pernyataan Penggunaan AI **68**

Bab I: Latar Belakang

1.1 Latar Belakang

Dokumen ini adalah implementasi dari tugas besar mata kuliah IF3070 Dasar Intelegensi Artifisial, dimana dilakukan pengimplementasian algoritma *local search* dalam membuat keputusan sesuai studi kasus yang diberikan pada spesifikasi. Adapun studi kasus yang diberikan adalah sebagai berikut:

> *"Anda adalah seorang Forward Deployed Engineer (FDE) di PT ~~Owen~~ Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.*
>
> *Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.*
>
> *Tim operasional meminta Anda membangun sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal."*

Dengan pemberian kasus tersebut, telah dirancang variabel, spesifikasi, dan juga solusi penyelesaian berdasarkan ketentuan dokumen, yaitu dengan pengimplementasian algoritma local search: hill-climbing, simulated annealing, dan genetic algorithm.

Bab II: Pembahasan

2.1 Penentuan Variabel dan Spesifikasi

Telah kami tentukan variabel terkait representasi kendaraan yang dapat diangkut beserta spesifikasi kapal dalam penyelesaian studi kasus. Terdapat tiga jenis kendaraan yang dapat dimuat dalam kapal, yaitu mobil, minibus, dan truk. Untuk ukuran dan spesifikasi masing-masing sebagai berikut:

Entitas	Atribut Data	Keterangan
vehicle	id	“mobil”; “minibus”; “truk”
	dimensions (x, y)	mobil = (1, 2); minibus = (1, 3); truk = (2, 3)
	orientation	horizontal/vertikal
	shippingFee (juta rupiah)	mobil = 20; minibus = 30; truk = 60
	weight (ton)	mobil = 5; minibus = 12; truk = 30
	eta (hari)	integer, 2
vehicleStock	unit	mobil = 20; minibus = 7; truk = 5

Adapun untuk spesifikasi ukuran dan berat maksimal yang dapat diangkut dari kapal tempat kendaraan-kendaraan tersebut akan diangkut adalah sebagai berikut:

Entitas	Atribut Data	Keterangan
ship	height, width (x, y)	(7, 5)
	maxCapacity (ton)	100

Untuk masing-masing implementasi kode pada algoritma, atribut data tersimpan dalam variabel yang dapat diubah pada inisialisasi awal. Maka dari itu, atribut data diatas hanya merepresentasikan bentuk default dari permasalahan yang telah disetujui oleh kelompok dalam mengerjakan algoritma. Apabila ada perubahan tertentu terkait ukuran atau bentuk dari kapal ataupun kendaraan, dapat disesuaikan demikian.

2.2 Penyelesaian dengan Hill Climbing

Hill climbing adalah salah satu local search dengan pencarian seakan-akan sedang mendaki gunung berkabut tebal dalam keadaan amnesia. Hill climbing memiliki beberapa jenis, tetapi yang digunakan dalam tugas ini adalah hill climbing steepest ascent. Pada steepest ascent, neighbor diambil jika valuenya lebih besar dari value sekarang. Prosesnya dimulai dengan state awal di-generate secara random. Kemudian loop dijalankan untuk membangkitkan suksesor, memilih suksesor terbaik untuk dijadikan neighbor. Value sekarang dibandingkan dengan value neighbor. Jika value neighbor lebih baik daripada value sekarang, maka state akan berpindah ke neighbor dan kemudian membangkitkan suksesor lagi. Sedangkan jika value neighbor kurang atau sama dengan value sekarang, maka loop akan berhenti.

Steepest ascent memiliki beberapa kelebihan. Prosesnya lebih cepat, hemat memori, dan lebih sederhana karena hanya berpindah state jika value neighbor lebih besar. Bagaimanapun, hill climbing juga memiliki kekurangan, yaitu besar kemungkinan hasilnya terjebak di local maximum, flat local maximum, atau shoulder. Padahal, bisa saja ada global maximum yang belum diperiksa. Maka dari itu, hasil hill climbing sangat bergantung dengan initial state yang ditetapkan secara random di awal.

2.2.1 Implementasi Algoritma

Pada algoritma ini, dibuat sebuah class bernama Vehicle yang berisi atribut tetap dan atribut yang berubah. Seperti yang telah dijelaskan pada bagian 2.1, terdapat total 32 objek kendaraan yang digunakan. State direpresentasikan dengan kumpulan 32 objek tersebut, dengan sebagian berada di dalam dan luar geladak kapal.

Aturan koordinat yang digunakan adalah titik (0, 0) berada di pojok kiri atas, dengan matriks 5 baris x 7 kolom. Titik row/column yang dimiliki oleh suatu kendaraan adalah titik pojok kiri atas kendaraan.

Objective function menggunakan total shippingFee dari kendaraan yang berada di dalam geladak kapal (insideShip = True).

Penjelasan tiap fungsi:

Deskripsi	Source Code

get_cells: Digunakan untuk mendapatkan daftar sel yang ditempati oleh suatu kendaraan	`def get_cells(self):`
	`  if (self.orientation == "H"):`
	`  rowCount = self.width`
	`  columnCount = self.length`
	`  else:`
	`  rowCount = self.length`
	`  columnCount = self.width`
	
	`  cells = []`
	`  for r in range(rowCount):`

	``
	            for c in range(columnCount):
	                cells.append((self.row + r, self.column + c))
	
	        return cells
	``

calculate_value(vehicles): Digunakan untuk menghitung total value dalam suatu state, di mana hanya shippingFee kendaraan yang berada di geladak yang dihitung.	```
	def calculate_value(vehicles):
	    totalShippingFee = 0
	    for v in vehicles:
	        # jika kendaraan di luar geladak, tidak perlu dihitung
	        if(v.insideShip == False):
	            continue
	        else:
	            totalShippingFee += v.shippingFee
	
	    return totalShippingFee
	``

create_vehicles(): Digunakan untuk membuat objek kendaraan di awal dengan stock yang telah ditentukan pada bagian 2.1. Fungsi ini menghasilkan vehicles (list kendaraan beserta atributnya)	```
	def create_vehicles():
	    vehicles = []
	
	    # memasukkan mobil dengan stock 20
	    for i in range(1, 21):
	        vehicles.append(Vehicle("mobil_" + str(i), "mobil", 1, 2, 20, 5, 2))
	
	    # memasukkan minibus dengan stock 7
	    for i in range (1, 8):
	        vehicles.append(Vehicle("minibus_" + str(i), "minibus", 1, 3, 30, 12, 2))
	
	    # memasukkan truk dengan stock 5
	    for i in range(1, 6):
	        vehicles.append(Vehicle("truk_" + str(i), "truk", 2, 3, 60, 30, 2))
	
	    return vehicles
	``

create_initial_state(vehicle) : Digunakan untuk membuat state awal. Vehicles yang sudah dibuat oleh create_vehicles dimodifikasi atributnya sehingga terdapat kendaraan yang masuk ke dalam geladak kapal. Penentuan objek yang dimasukkan beserta posisi dan orientasinya ditentukan secara random hingga geladak penuh.	```
	def create_initial_state(vehicles):
	    failCount = 0
	    maxFail = 50 # maxFail berfungsi untuk memberhentikan looping saat geladak sudah dianggap penuh
	
	    while(failCount < maxFail):
	        vhcOutsideShip = []
	        for v in vehicles:
	            if(v.insideShip == False):
	                vhcOutsideShip.append(v)
	        if(vhcOutsideShip == []):
	            break
	``

	``
	        choosenVehicle = random.choice(vhcOutsideShip)
	        choosenVehicle.insideShip = True
	        choosenVehicle.row = random.randint(0, shipWidth - 1)
	        choosenVehicle.column = random.randint(0, shipLength - 1)
	        choosenVehicle.orientation = random.choice(["H", "V"])
	        if(is_valid(vehicles)):
	            failCount = 0
	        else:
	            choosenVehicle.insideShip = False
	            choosenVehicle.row = None
	            choosenVehicle.column = None
	            choosenVehicle.orientation = "H"
	            failCount += 1
	
	    return vehicles
	``

is_valid(vehicles):
Digunakan untuk mengecek apakah suatu suksesor/state valid atau tidak. Dilakukan pengecekan apakah kendaraan berada sepenuhnya di dalam geladak, apakah suatu sel sudah ditempati, dan apakah totalWeight masih memenuhi maxCapacity.
	``
	def is_valid(vehicles):
	    totalWeight = 0
	    cellUsedList = []
	
	    for v in vehicles:
	        if(v.insideShip == False):
	            continue
	        else:
	            cells = v.get_cells()
	
	            for c in cells:
	                # cek apakah sel tersebut di dalam atau luar geladak
	                if(c[0] < 0 or c[0] > shipWidth-1 or c[1] < 0 or c[1] > shipLength - 1):
	                    return False
	
	                # cek apakah sel tersebut udah pernah dipake
	                if(c in cellUsedList):
	                    return False
	                else:
	                    # klo belum pernah, ditambah ke list
	                    cellUsedList.append(c)
	
	            # menghitung total weight
	            totalWeight += v.weight
	            # cek apakah total weight melebih max capacity
	            if(totalWeight > shipMaxCapacity):
	                return False
	
	    return True
	``

change_orientation(vehicle	```
	def change_orientation(vehicles, index):
	``

s, index): Digunakan untuk menjalankan move mengubah orientasi dari horizontal ke vertikal dan sebaliknya.	`copyVehicles = copy.deepcopy(vehicles)`
	
	`v = copyVehicles[index]`
	`if(v.orientation == "H"):`
	`v.orientation = "V"`
	`elif(v.orientation == "V"):`
	`v.orientation = "H"`
	
	`return copyVehicles`

move_vehicle(vehicles, index, newRow, newColumn): Digunakan untuk memindahkan suatu kendaraan ke titik lainnya pada kapal.	`def move_vehicle(vehicles, index, newRow, newColumn):`
	`copyVehicles = copy.deepcopy(vehicles)`
	
	`v = copyVehicles[index]`
	
	`v.row = newRow`
	`v.column = newColumn`
	
	`return copyVehicles`

swap_vehicle(vehicles, index1, index2): Digunakan untuk menukar kendaraan yang berada di geladak dengan kendaraan lain yang berada di dalam maupun luar geladak. Ketentuan yang ditetapkan adalah, jika kendaraan 2 berada di dalam geladak, maka tidak hanya titik, tetapi orientasinya juga ikut ditukar untuk memaksimalkan hasil. Sedangkan jika ditukar dengan kendaraan di luar, maka kendaraan di luar mengikuti titik dan orientasi kendaraan di dalam.	`def swap_vehicle(vehicles, index1, index2):`
	`copyVehicles = copy.deepcopy(vehicles)`
	`tempRow1 = None`
	`tempColumn1 = None`
	`tempOrientation1 = None`
	
	`v1 = copyVehicles[index1]`
	`v2 = copyVehicles[index2]`
	`if(v1.insideShip == True and v2.insideShip == False):`
	`#jika ditukar dengan kendaraan di luar geladak`
	`v2.row = v1.row`
	`v2.column = v1.column`
	`v2.orientation = v1.orientation`
	`v2.insideShip = True`
	
	`# membuat v1 menjadi default (karena dikeluarkan)`
	`v1.row = None`
	`v1.column = None`
	`v1.orientation = "H"`
	`v1.insideShip = False`
	
	`elif(v1.insideShip == True and v2.insideShip == True):`
	`#jika ditukar dengan kendaraan lain di dalam geladak`
	`tempRow1 = v1.row`
	`tempColumn1 = v1.column`
	`tempOrientation1 = v1.orientation`
	
	`# untuk kasus ini, saya buat agar orientasi juga ditukar untuk menurunkan kemungkinan overlap`

	``` v1.row = v2.row v1.column = v2.column v1.orientation = v2.orientation v2.row = tempRow1 v2.column = tempColumn1 v2.orientation = tempOrientation1 return copyVehicles ```

successor_maker(vehicles): Digunakan untuk membangkitkan semua suksesor dari state sebelumnya. Loop dijalankan ke masing-masing kendaraan di dalam geladak dan setiap kendaraan menjalankan 3 move yang telah didefinisikan pada fungsi sebelumnya.	```
	def successor_maker(vehicles):
	    successors = []
	
	    for i in range(len(vehicles)):
	        # kondisional buat memastikan yang diubah hanya kendaraan yang di dalam geladak saja
	        if(vehicles[i].insideShip == False):
	            continue
	
	        # ubah orientasi
	        newOrien = change_orientation(vehicles, i)
	        if(is_valid(newOrien)):
	            successors.append(newOrien)
	
	        # pindah
	        for y in range(shipWidth): # cek tiap baris
	            for x in range(shipLength): # cek tiap kolom
	                # kalo koordinat tujuan sama dengan koordinat sekarang, skip
	                if(vehicles[i].row == y and vehicles[i].column == x):
	                    continue
	                else:
	                    newMove = move_vehicle(vehicles, i, y, x)
	                    if(is_valid(newMove)):
	                        successors.append(newMove)
	
	        # tukar
	        for v in range(len(vehicles)):
	            # kalo kendaraan awal sama kendaraan yg mau dituker itu sama, skip aja
	            if(i == v):
	                continue
	            else:
	                newSwap = swap_vehicle(vehicles, i, v)
	                if(is_valid(newSwap)):
	                    successors.append(newSwap)
	
	    return successors
	``

steepest_ascent(vehicles): Digunakan untuk menentukan apakah value neighbor lebih baik atau tidak jika dibandingkan dengan current value dan menghasilkan state selanjutnya yang sesuai.	`def steepest_ascent(vehicles):`
	`    startTime = time()`
	`    current = vehicles`
	`    currentValue = calculate_value(vehicles)`
	`    iterations = 0`
	`    history = [currentValue]`
	
	`    while True:`
	`        successors = successor_maker(current)`
	`        if(successors == []):`
	`            break`
	
	`        bestSucc = None`
	`        bestValue = -1`
	`        for s in successors:`
	`            successorValue = calculate_value(s)`
	`            if(successorValue > bestValue):`
	`                bestValue = successorValue`
	`                bestSucc = s`
	
	`        if bestValue <= currentValue:`
	`            break`
	`        else:`
	`            current = bestSucc`
	`            currentValue = bestValue`
	`            iterations += 1   # penghitungan iterasi tidak termasuk iterasi terakhir yang gagal`
	`            history.append(currentValue)`
	
	`    duration = time() - startTime`
	`    return current, iterations, duration, history`

print_matrix(matrix): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk mencetak kondisi kapal.	`def print_matrix(matrix: dict) -> None:`
	`    cellWidth = max(2, len(str(max(matrix.values()))))`
	`    segment = "─" * (cellWidth + 2)`
	`    # Menggunakan unicode agar tabel menjadi rapih`
	`    print("┌" + "┬".join([segment] * shipLength) + "┐")`
	`    for i in range(shipWidth):`
	`        cells = [f" {str(matrix[(i, j)]).rjust(cellWidth)} " for j in range(shipLength)]`
	`        print("│" + "│".join(cells) + "│")`
	`        if i < shipWidth - 1:`
	`            print("├" + "┼".join([segment] * shipLength) + "┤")`
	`    print("└" + "┴".join([segment] * shipLength) + "┘")`

divider(title, width, char): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk mencetak garis pemisah antara satu bagian	`def divider(title: str = "", width: int = 60, char: str = "=") -> None:`
	`    if title == "":`
	`        print(char * width)`
	`        return`
	`    label = f" {title} "`

dengan bagian lainnya.	`    left = (width - len(label)) // 2`
	`    right = width - len(label) - left`
	`    print()`
	`    print(char * left + label + char * right)`
	`    print()`

loading(message, seconds): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk menampilkan pesan dan animasi loading sebelum menampilkan info baru.	`def loading(message: str, seconds: float = 1.5) -> None:`
	`    global delayTime`
	`    startLoading = perf_counter()`
	`    frames = ["   ", ".  ", ".. ", "..."]`
	`    endTime = time() + seconds`
	`    i = 0`
	`    while time() < endTime:`
	`        print(f"\r{message}{frames[i % 4]}", end="",`
	`flush=True)`
	`        sleep(0.3)`
	`        i += 1`
	`    print(f"\r{message}...")`
	`    delayTime += perf_counter() - startLoading`

build_tables(title, headers, rows): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk membuat tabel informasi.	`def build_table(title: str, headers: tuple, rows: list) ->`
	`list:`
	`    codeW = max([len(headers[0])] + [len(str(c)) for c, _ in`
	`rows])`
	`    typeW = max([len(headers[1])] + [len(t) for _, t in rows])`
	`    totalW = codeW + typeW + 7`
	`    border = lambda l, m, r: l + "─" * (codeW + 2) + m + "─" *`
	`(typeW + 2) + r`
	`    lines = [`
	`        title.center(totalW),`
	`        border("┌", "┬", "┐"),`
	`        f"│ {headers[0].ljust(codeW)} │`
	`{headers[1].ljust(typeW)} │",`
	`        border("├", "┼", "┤"),`
	`    ]`
	`    for c, t in rows:`
	`        lines.append(f"│ {str(c).rjust(codeW)} │`
	`{t.ljust(typeW)} │")`
	`    if not rows:`
	`        lines.append(f"│ {'-'.center(codeW)} │`
	`{'-'.center(typeW)} │")`
	`    lines.append(border("└", "┴", "┘"))`
	`    return lines`

print_vehicle_tables(placed, unplaced): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk menampilkan tabel berisi informasi apakah suatu	`def print_vehicle_tables(placed: list, unplaced: list) -> None:`
	`    left = build_table("PLACED", ("Code", "Type"), placed)`
	`    right = build_table("UNPLACED", ("Jumlah", "Jenis"),`
	`unplaced)`
	`    height = max(len(left), len(right))`
	`    while len(left) < height:`
	`        left.append(" " * len(left[0]))`

kendaraan berada di dalam atau luar geladak.	```
	    while len(right) < height:
	        right.append(" " * len(right[0]))
	    for a, b in zip(left, right):
	        print(a + " " + b)
	``

build_ship_matrix(vehicles): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk menampilkan kondisi geladak kapal.	```
	def build_ship_matrix(vehicles):
	    matrix = {(i, j): 0 for i in range(shipWidth) for j in range(shipLength)}
	    number = 1
	
	    for v in vehicles:
	        if v.insideShip == False:
	            continue
	        cells = v.get_cells()
	        for c in cells:
	            matrix[c] = number
	        number += 1
	
	    return matrix
	``

build_vehicle_lists(vehicles) : Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk menerjemahkan state ke list placed dan unplaced.	```
	def build_vehicle_lists(vehicles):
	    placed = []
	    counter = {"mobil" : 0, "minibus" : 0, "truk" : 0}
	    number = 1
	
	    for v in vehicles:
	        if v.insideShip:
	            placed.append((number, v.vhcType))
	            number += 1
	        else:
	            counter[v.vhcType] += 1
	
	    unplaced = []
	    for jenis in counter:
	        if counter[jenis] > 0:
	            unplaced.append((f"{counter[jenis]}x", jenis))
	
	    return placed, unplaced
	``

plot_history(history, height): Diadaptasi dari potongan kode genetic.py milik Azza (18224094). Digunakan untuk membuat grafik yang menggambarkan hubungan antara banyak iterasi dengan value yang didapatkan.	```
	def plot_history(history, height: int = 12) -> None:
	    low = min(history)
	    high = max(history)
	    if high == low:
	        high = low + 1
	    columnGap = 6 if len(history) <= 12 else 4
	    width = len(history) * columnGap
	
	    grid = [[" "] * width for _ in range(height + 1)]
	    for idx in range(len(history)):
	        x = idx * columnGap + 2
	``

	``
	        y = round((history[idx] - low) / (high - low) * height)
	        grid[height - y][x] = "*"
	    print("Grafik value terhadap iterasi (* = value)")
	    for row in range(height + 1):
	        value = high - (high - low) * row / height
	        print(f"{value:7.1f} |" + "".join(grid[row]))
	    print(" └" + "─" * width)
	    labels = "".join(str(i).ljust(columnGap) for i in range(len(history)))
	    print(" " + labels + " (iterasi)")
	``

main(): Digunakan untuk menjalankan keseluruhan program, mulai dari membuat objek kendaraan, membuat initial state, membuat suksesor, menentukan neighbor, membandingkan value, hingga menampilkan hasil algoritma.	```
	def main():
	    numberRuns= 3
	    results = []
	
	    for run in range(numberRuns):
	        divider(f"RUN {run + 1}")
	
	        loading("Membuat state awal", 2)
	        vehicles = create_vehicles() # buat kendaraan
	        vehicles = create_initial_state(vehicles) # beberapa masu ke geladak sebagai initial state
	        initialState = copy.deepcopy(vehicles)
	
	        divider("STATE AWAL")
	        matrix = build_ship_matrix(initialState)
	        placed, unplaced = build_vehicle_lists(initialState)
	
	        loading("Membuat matriks", 2)
	        print_matrix(matrix)
	
	        loading("Membuat daftar kendaraan yang masuk geladak dan yang di luar", 2)
	        print_vehicle_tables(placed, unplaced)
	
	        loading("Menjalankan steepest ascent", 2)
	        endState, iterations, duration, history = steepest_ascent(vehicles)
	
	        divider("STATE AKHIR")
	        matrix = build_ship_matrix(endState)
	        placed, unplaced = build_vehicle_lists(endState)
	        print_matrix(matrix)
	        print_vehicle_tables(placed, unplaced)
	
	        loading("Ringkasan run ke-" + str(run+1), 2)
	        print(f"Value awal : {history[0]}")
	        print(f"Value akhir : {history[-1]}")
	        print(f"Banyak iterasi : {iterations}")
	        print(f"Durasi : {duration:.2f} detik")
	        plot_history(history)
	``

```
        results.append({
            "initialState" : initialState,
            "endState" : endState,
            "startValue" : history[0],
            "endValue" : history[-1],
            "iterations" : iterations,
            "duration" : duration,
            "history" : history,
        })

    divider("RINGKASAN EKSPERIMEN")
    print(f"{'Run':<6}{'Value awal':>12}{'Value
akhir':>13}{'Iterasi':>9}{'Durasi (s)':>12}")
    for i in range(len(results)):
        r = results[i]
        print(f"{i +
1:<6}{r['startValue']:>12}{r['endValue']:>13}{r['iterations']:>
9}{r['duration']:>12.2f}")
```

2.2.2 Hasil Algoritma

Algoritma dijalankan sebanyak tiga kali dalam satu kali run. Setiap run dimulai dari pembangkitan state acak yang baru, dengan mengisi geladak terus menerus hingga fail mencapai maxFail = 50. Durasi algoritma diukur di dalam fungsi steepest ascent, tanpa menghitung durasi saat menampilkan di CLI (loading). Hasil algoritma dapat dilihat pada repository github output/output.hill_climbing/output.hill_climbing.txt.

Ringkasan hasil run:

Running ke-1	State Awal

Ringkasan run ke-1:
Value awal : 270
Value akhir : 270
Banyak iterasi : 0
Durasi : 0.18 detik

```
Grafik value terhadap iterasi   (* = value)
  271.0 |
  270.9 |
  270.8 |
  270.8 |
  270.7 |
  270.6 |
  270.5 |
  270.4 |
  270.3 |
  270.2 |
  270.1 |
  270.0 |   *
        +-------
         0      (iterasi)
```

```
Membuat matriks...
+---+---+---+----+----+----+---+
| 5 | 7 | 0 | 10 | 10 | 10 | 0 |
+---+---+---+----+----+----+---+
| 5 | 7 | 2 | 10 | 10 | 10 | 0 |
+---+---+---+----+----+----+---+
| 6 | 7 | 2 | 0  | 0  | 9  | 4 |
+---+---+---+----+----+----+---+
| 6 | 1 | 8 | 8  | 8  | 9  | 4 |
+---+---+---+----+----+----+---+
| 0 | 1 | 0 | 3  | 3  | 9  | 0 |
+---+---+---+----+----+----+---+
Membuat daftar kendaraan yang masuk geladak dan yang di luar...
      PLACED                UNPLACED
+------+---------+      +--------+--------+
| Code | Type    |      | Jumlah | Jenis  |
+------+---------+      +--------+--------+
|    1 | mobil   |      |    14x | mobil  |
|    2 | mobil   |      |     4x | minibus|
|    3 | mobil   |      |     4x | truk   |
|    4 | mobil   |      +--------+--------+
|    5 | mobil   |
|    6 | mobil   |
|    7 | minibus |
|    8 | minibus |
|    9 | minibus |
|   10 | truk    |
+------+---------+
```

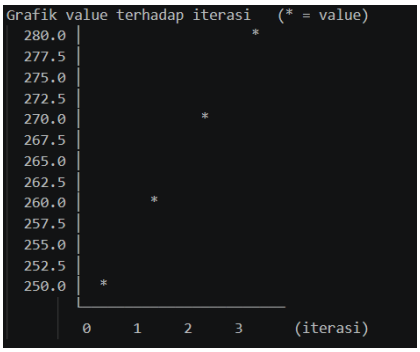
State Akhir

```
+---+---+---+----+----+----+---+
| 5 | 7 | 0 | 10 | 10 | 10 | 0 |
+---+---+---+----+----+----+---+
| 5 | 7 | 2 | 10 | 10 | 10 | 0 |
+---+---+---+----+----+----+---+
| 6 | 7 | 2 | 0  | 0  | 9  | 4 |
+---+---+---+----+----+----+---+
| 6 | 1 | 8 | 8  | 8  | 9  | 4 |
+---+---+---+----+----+----+---+
| 0 | 1 | 0 | 3  | 3  | 9  | 0 |
+---+---+---+----+----+----+---+
      PLACED                UNPLACED
+------+---------+      +--------+--------+
| Code | Type    |      | Jumlah | Jenis  |
+------+---------+      +--------+--------+
|    1 | mobil   |      |    14x | mobil  |
|    2 | mobil   |      |     4x | minibus|
|    3 | mobil   |      |     4x | truk   |
|    4 | mobil   |      +--------+--------+
|    5 | mobil   |
|    6 | mobil   |
|    7 | minibus |
|    8 | minibus |
|    9 | minibus |
|   10 | truk    |
+------+---------+
```

Running ke-2

State Awal

Ringkasan run ke-2:
Value awal : 250
Value akhir : 280
Banyak iterasi : 3
Durasi : 0.78 detik



11	11	0	0	8	8	7
10	0	6	6	3	0	7
10	0	1	1	3	0	12
2	9	0	5	5	0	12
2	9	0	4	4	0	12

Membuat daftar kendaraan yang masuk geladak dan yang di luar...

PLACED

Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	mobil
6	mobil
7	mobil
8	mobil
9	mobil
10	mobil
11	mobil
12	minibus

UNPLACED

Jumlah	Jenis
9x	mobil
6x	minibus
5x	truk

State Akhir

11	11	11	0	6	6	5
8	0	4	4	3	0	5
8	0	1	1	3	0	12
2	7	0	10	10	10	12
2	7	0	9	9	9	12

PLACED

Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	mobil
6	mobil
7	mobil
8	mobil
9	minibus
10	minibus
11	minibus
12	minibus

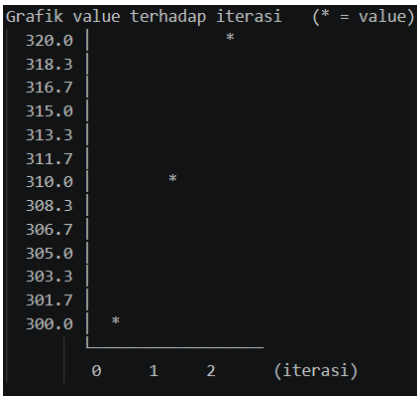
UNPLACED

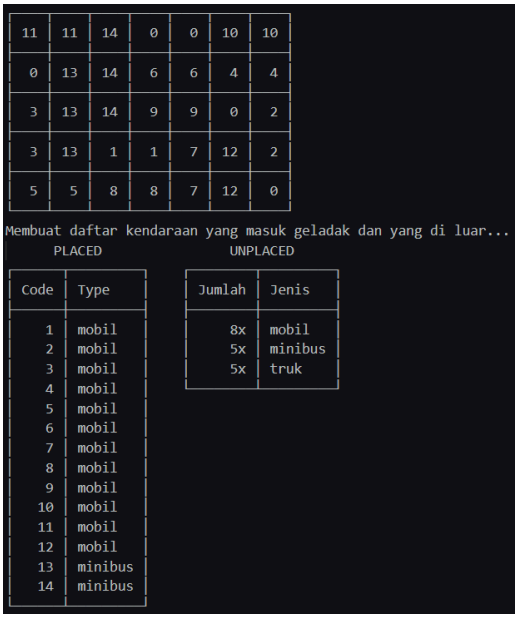
Jumlah	Jenis
12x	mobil
3x	minibus
5x	truk

Running ke-3

State Awal

Ringkasan run ke-3:
Value awal : 300
Value akhir : 320
Banyak iterasi : 2
Durasi : 0.69 detik





11	11	14	0	0	10	10
0	13	14	6	6	4	4
3	13	14	9	9	0	2
3	13	1	1	7	12	2
5	5	8	8	7	12	0

Membuat daftar kendaraan yang masuk geladak dan yang di luar...

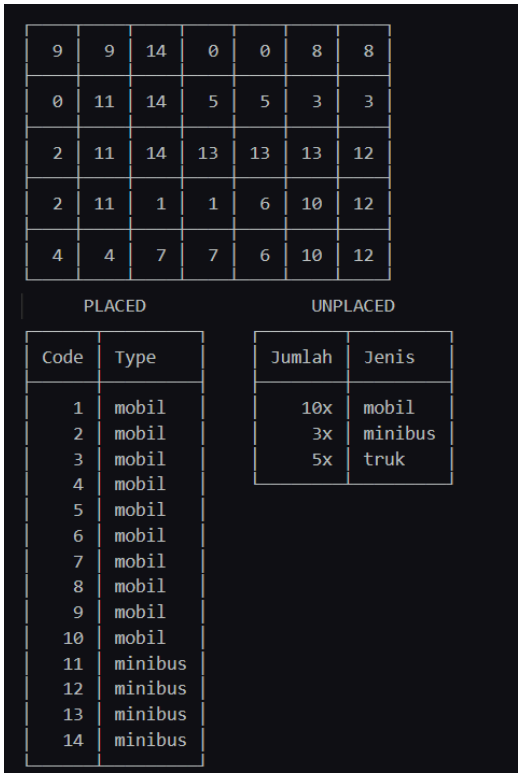
PLACED

Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	mobil
6	mobil
7	mobil
8	mobil
9	mobil
10	mobil
11	mobil
12	mobil
13	minibus
14	minibus

UNPLACED

Jumlah	Jenis
8x	mobil
5x	minibus
5x	truk

State Akhir



9	9	14	0	0	8	8
0	11	14	5	5	3	3
2	11	14	13	13	13	12
2	11	1	1	6	10	12
4	4	7	7	6	10	12

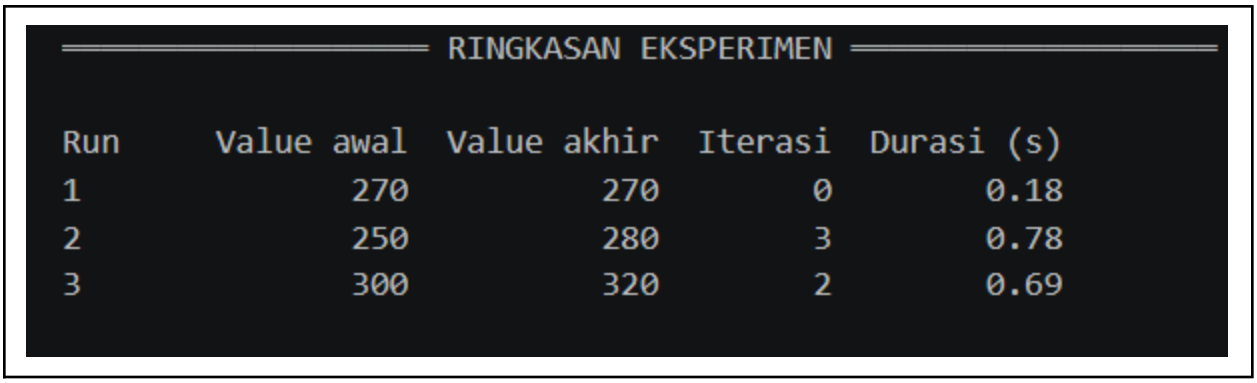
PLACED

Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	mobil
6	mobil
7	mobil
8	mobil
9	mobil
10	mobil
11	minibus
12	minibus
13	minibus
14	minibus

UNPLACED

Jumlah	Jenis
10x	mobil
3x	minibus
5x	truk

Ringkasan Keseluruhan



RINGKASAN EKSPERIMEN

Run	Value awal	Value akhir	Iterasi	Durasi (s)
1	270	270	0	0.18
2	250	280	3	0.78
3	300	320	2	0.69

Dari hasil tersebut, dapat dianalisis terkait beberapa hal. Setelah menjalankan run 1, 2, dan 3, terlihat bahwa value akhir cukup dekat dengan batas atas, yaitu 350 (35 sel x 10 juta per sel). Walaupun begitu, sulit untuk benar-benar mencapai global maximum karena sering berhenti di local optimum ketika tidak ada suksesor yang lebih baik.

Selain itu, dapat dilihat juga bahwa setiap run memiliki hasil yang berbeda dan tidak konsisten. Hal ini lagi-lagi disebabkan oleh risiko berhenti di local optimum besar, sehingga muncul kemungkinan value stuck pada nilai yang rendah dan tidak bisa mendapatkan nilai yang lebih tinggi.

Karena adanya batasan pemilihan neighbor yang harus memilih nilai yang lebih tinggi, maka proses pencarian juga selesai lebih cepat. Program tidak melanjutkan pencarian ketika diketahui bahwa sudah tidak ada lagi nilai yang lebih baik. Bisa dilihat juga pada ringkasan eksperimen bahwa durasi yang dibutuhkan sangat cepat, berkisar 0.18 s - 0.69 s.

State awal pada iterasi steepest ascent sangat berpengaruh pada hasil. Jika state awal sudah berada di local optimum atau jurang, maka akan sulit untuk melanjutkan pencarian nilai yang lebih baik. Tetapi, jika kebetulan state awal langsung mendapatkan global maximum, maka hasilnya tentu akan berbeda drastis.

Kondisi “macet” sering terjadi pada steepest ascent. Alasannya adalah karena aksi pindah dan putar sulit mendapatkan hasil yang valid karena rawan tertabrak dengan kendaraan lain atau posisi kendaraan tidak sepenuhnya berada di dalam geladak. Sedangkan tukar dalam-luar memiliki kemungkinan valid yang lebih tinggi, didukung dengan orientasi yang mengikuti orientasi kendaraan di dalam.

Jika steepest ascent hill climbing dibandingkan dengan algoritma local search lainnya, algoritma ini termasuk yang paling ringan dan sederhana. Pencariannya pun jauh lebih cepat. Tetapi, hasilnya sangat bergantung pada initial state, sehingga sulit untuk mendapatkan hasil yang benar-benar optimal.

2.3 Penyelesaian dengan Simulated Annealing

Simulated annealing (SA) adalah algoritma local search yang terinspirasi dari proses annealing pada metalurgi, yaitu logam dipanaskan hingga suhu tinggi lalu didinginkan secara perlahan agar strukturnya mencapai keadaan yang stabil. Seperti hill-climbing, SA hanya menyimpan satu current state dan pada setiap iterasi membangkitkan satu neighbor secara acak. Perbedaannya, SA tidak hanya menerima neighbor yang lebih baik: neighbor yang lebih buruk tetap dapat diterima dengan probabilitas e^(ΔE/T), yang bergantung pada seberapa buruk neighbor tersebut (ΔE) dan suhu saat itu (T). Suhu diturunkan secara bertahap menurut jadwal pendinginan, sehingga pada awal pencarian SA bersifat eksploratif dan pada akhir pencarian berperilaku seperti hill-climbing.

Kelebihan SA adalah kemampuannya keluar dari local maximum, flat local maximum, dan shoulder, karena langkah yang menurunkan value masih dapat diterima ketika suhu tinggi. Kekurangannya, SA membutuhkan jauh lebih banyak iterasi dibandingkan hill-climbing, dan kualitas hasilnya sangat bergantung pada parameter jadwal pendinginan (suhu awal, laju pendinginan, dan suhu minimum). Selain itu, sesuai pseudocode, SA mengembalikan state pada iterasi terakhir, sehingga state terbaik yang pernah dikunjungi dapat terlewat.

Algoritma simulated annealing (SA) diimplementasikan pada file src/simulated_annealing.py (branch sean-simmulatedannealing) dengan mengacu pada pseudocode simulated annealing di salindia kuliah. Berbeda dengan genetic algorithm yang menyimpan urutan jenis kendaraan sebagai genome, SA bekerja pada satu state yang secara eksplisit menyimpan posisi dan orientasi setiap kendaraan, sehingga state selalu merepresentasikan kondisi kapal yang sebenarnya.

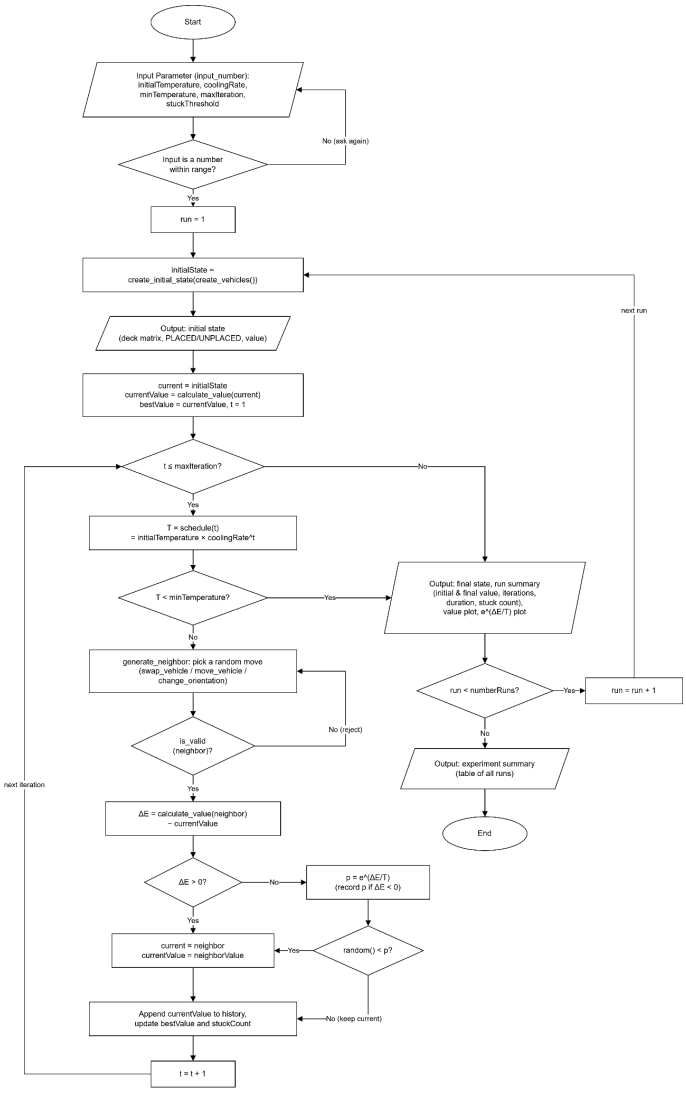
State direpresentasikan sebagai list berisi 32 kendaraan (20 mobil, 7 minibus, dan 5 truk sesuai vehicleStock). Setiap kendaraan adalah dictionary yang menyalin atribut jenisnya dari vehicles (dimensions, orientation, shippingFee, weight, eta), lalu ditambah atribut id unik (misalnya “mobil_4”), type (jenis kendaraan), dan position.

```
{"id": "mobil_4", "type": "mobil", "dimensions": (2, 1), "orientation":
"vertical", "shippingFee": 20, "weight": 5, "eta": 2, "position": (0, 3)} #
di dalam kapal, menempati sel (0,3) dan (1,3)

{"id": "truk_1", "type": "truk", "dimensions": (2, 3), "orientation":
"horizontal", "shippingFee": 60, "weight": 30, "eta": 2, "position": None} #
di luar kapal
```

Alur fungsi simulated_annealing mengikuti pseudocode salindia, dengan tiga penyesuaian agar dapat dijalankan sebagai program. Pertama, kondisi berhenti “if T = 0” diganti menjadi T<minTemperature, karena dengan jadwal geometrik suhu tidak pernah tepat bernilai 0; di komputer suhu baru menjadi 0 akibat underflow setelah sekitar 744 ribu iterasi, padahal sejak T = 0,01 peluang menerima neighbor yang lebih buruk sudah praktis 0. Kedua, perulangan “for t = 1 to ∞” dibatasi oleh maxIteration sebagai batas pengaman agar program pasti berhenti. Ketiga,

“a randomly selected successor of current” diwujudkan oleh generate_neighbor yang hanya mengembalikan successor yang valid, karena pseudocode mengasumsikan setiap successor merupakan state yang sah. Initial state (problem.INITIAL-STATE) dibangkitkan secara acak pada fungsi main sebelum simulated_annealing dipanggil, sehingga state awal dapat ditampilkan dan durasi hanya mengukur proses pencarian. Pencatatan riwayat value, riwayat e^(ΔE/T), frekuensi stuck, dan durasi merupakan tambahan untuk kebutuhan eksperimen dan tidak memengaruhi keputusan menerima atau menolak neighbor.



2.3.1 Implementasi Algoritma

Deskripsi	Source Code

create_vehicles: Digunakan untuk membuat 32 kendaraan (20 mobil, 7 minibus, dan 5 truk sesuai vehicleStock) dengan id unik. Seluruh kendaraan mula-mula berada di luar kapal (position = None).	`def create_vehicles() -> shipState:`
	`  fleet = []`
	`  for vehicleCode in (1, 2, 3):`
	`  for k in range(vehicleStock[vehicleCode]):`
	`  vehicle = vehicles[vehicleCode].copy()`
	`  vehicle["id"] =`
	`f"{vehicles[vehicleCode]['id']}_{k + 1}"`
	`  vehicle["type"] = vehicles[vehicleCode]["id"]`
	`  vehicle["position"] = None`
	`  fleet.append(vehicle)`
	`  return fleet`

rotate_vehicle: Digunakan untuk mengembalikan salinan kendaraan dengan dimensi ditukar dan orientasi dibalik (horizontal ke vertikal), sama dengan genetic_algorithm.py.	`def rotate_vehicle(vehicle: dict) -> dict:`
	`  rotatedVehicle = vehicle.copy()`
	`  rotatedVehicle["dimensions"] =`
	`(vehicle["dimensions"][1], vehicle["dimensions"][0])`
	`  rotatedVehicle["orientation"] = "vertical" if`
	`vehicle["orientation"] == "horizontal" else "horizontal"`
	`  return rotatedVehicle`

get_cells: Digunakan untuk mendapatkan seluruh sel geladak yang ditempati sebuah kendaraan berdasarkan position (pojok kiri atas) dan dimensions.	`def get_cells(vehicle: dict) -> list:`
	`  positionX, positionY = vehicle["position"]`
	`  vehicleWidth = vehicle["dimensions"][0]`
	`  vehicleHeight = vehicle["dimensions"][1]`
	`  return [(positionX + i, positionY + j) for i in`
	`range(vehicleWidth) for j in range(vehicleHeight)]`

is_valid: Digunakan untuk memeriksa constraint sebuah state: seluruh sel berada di dalam batas geladak, tidak ada sel yang ditempati lebih dari satu kendaraan, dan total berat tidak melebihi shipMaxCapacity.	`def is_valid(currentState: shipState) -> bool:`
	`  occupiedCells = set()`
	`  totalWeight = 0`
	`  for vehicle in currentState:`
	`  if vehicle["position"] is None:`
	`  continue`
	`  totalWeight += vehicle["weight"]`
	`  if totalWeight > shipMaxCapacity:`
	`  return False`
	`  for (i, j) in get_cells(vehicle):`
	`  if i < 0 or i >= shipWidth or j < 0 or j >=`
	`shipHeight:`
	`  return False`
	`  if (i, j) in occupiedCells:`
	`  return False`
	`  occupiedCells.add((i, j))`
	`  return True`

calculate_value: Objective function, yaitu total shippingFee kendaraan yang berada di dalam kapal.	`def calculate_value(currentState: shipState) -> int:` `  return sum(vehicle["shippingFee"] for vehicle in currentState if vehicle["position"] is not None)`
calculate_weight: Digunakan untuk menghitung total berat kendaraan yang berada di dalam kapal.	`def calculate_weight(currentState: shipState) -> int:` `  return sum(vehicle["weight"] for vehicle in currentState if vehicle["position"] is not None)`

create_initial_state: Digunakan untuk membangkitkan initial state acak yang valid. Urutan kendaraan diacak, tiap kendaraan diberi orientasi dan koordinat acak, lalu dikembalikan ke luar kapal jika penempatannya melanggar constraint.	`def create_initial_state(currentState: shipState) -> shipState:`
	`  initialState = [vehicle.copy() for vehicle in currentState]`
	`  order = list(range(len(initialState)))`
	`  random.shuffle(order)`
	`  for index in order:`
	`  if random.random() < 0.5:`
	`  initialState[index] = rotate_vehicle(initialState[index])`
	`  initialState[index]["position"] = (random.randrange(shipWidth), random.randrange(shipHeight))`
	`  if not is_valid(initialState):`
	`  initialState[index]["position"] = None`
	`  return initialState`

swap_vehicle: Move 1. Digunakan untuk menukar position dua kendaraan acak (di dalam maupun di luar kapal) dengan orientasi masing-masing tetap. Pasangan dengan position yang sama tidak dipilih karena tidak mengubah state.	`def swap_vehicle(currentState: shipState) -> shipState:`
	`  neighbor = currentState.copy()`
	`  a = random.randrange(len(neighbor))`
	`  candidates = [b for b in range(len(neighbor)) if neighbor[b]["position"] != neighbor[a]["position"]]`
	`  if not candidates:`
	`  return neighbor`
	`  b = random.choice(candidates)`
	`  neighbor[a] = {**currentState[a], "position": currentState[b]["position"]}`
	`  neighbor[b] = {**currentState[b], "position": currentState[a]["position"]}`
	`  return neighbor`

move_vehicle: Move 2. Digunakan untuk memindahkan satu kendaraan acak ke koordinat	`def move_vehicle(currentState: shipState) -> shipState:`
	`  neighbor = currentState.copy()`
	`  index = random.randrange(len(neighbor))`
	`  destinations = [(i, j) for i in range(shipWidth) for j`

lain di geladak atau ke luar kapal (None).	`in range(shipHeight)] + [None]`
	`    destinations.remove(currentState[index]["position"])`
	`    neighbor[index] = {**currentState[index], "position": random.choice(destinations)}`
	`    return neighbor`

change_orientation: Move 3. Digunakan untuk memutar satu kendaraan acak (horizontal ke vertikal) dengan pojok kiri atas tetap.	`def change_orientation(currentState: shipState) -> shipState:`
	`    neighbor = currentState.copy()`
	`    index = random.randrange(len(neighbor))`
	`    neighbor[index] = rotate_vehicle(currentState[index])`
	`    return neighbor`

generate_neighbor: Digunakan untuk memilih salah satu dari tiga move secara acak dan mengulanginya hingga diperoleh neighbor yang valid.	`def generate_neighbor(currentState: shipState) -> shipState:`
	`    moves = [swap_vehicle, move_vehicle, change_orientation]`
	`    while True:`
	`        neighbor = random.choice(moves)(currentState)`
	`        if is_valid(neighbor):`
	`            return neighbor`

schedule: Digunakan untuk menghitung suhu pada iterasi ke-t dengan jadwal pendinginan geometrik T(t) = initialTemperature × coolingRate^t.	`def schedule(t: int, initialTemperature: float, coolingRate: float) -> float:` `    return initialTemperature * (coolingRate ** t)`

simulated_annealing: Digunakan untuk menjalankan satu run SA dari initial state sesuai pseudocode salindia. Mengembalikan state akhir, banyak iterasi, durasi, riwayat value, riwayat e^(ΔE/T), dan frekuensi stuck.	`def simulated_annealing(initialState: shipState, initialTemperature: float, coolingRate: float,`
	`                         minTemperature: float, maxIteration: int, stuckThreshold: int):`
	`    startTime = time()`
	`    current = initialState`
	`    currentValue = calculate_value(current)`
	`    bestValue = currentValue`
	`    history = [currentValue]`
	`    probabilityHistory = []`
	`    stuckCount = 0`
	`    iterationWithoutImprovement = 0`
	`    iterations = 0`
	
	`    for t in range(1, maxIteration + 1):`
	`        temperature = schedule(t, initialTemperature, coolingRate)`
	`        if temperature < minTemperature:`

	``
	            break
	        iterations = t
	
	        neighbor = generate_neighbor(current)
	        neighborValue = calculate_value(neighbor)
	        deltaE = neighborValue - currentValue
	
	        if deltaE > 0:
	            current, currentValue = neighbor, neighborValue
	        else:
	            probability = math.exp(deltaE / temperature)
	            # e^(deltaE/T) hanya dicatat saat deltaE < 0
	(saat deltaE = 0 nilainya selalu 1)
	            if deltaE < 0:
	                probabilityHistory.append((t, probability))
	            if random.random() < probability:
	                current, currentValue = neighbor,
	neighborValue
	        history.append(currentValue)
	
	        # Pencatatan frekuensi stuck di local optima
	        if currentValue > bestValue:
	            bestValue = currentValue
	            iterationWithoutImprovement = 0
	        else:
	            iterationWithoutImprovement += 1
	            if iterationWithoutImprovement ==
	stuckThreshold:
	                stuckCount += 1
	                iterationWithoutImprovement = 0
	
	    duration = time() - startTime
	    return current, iterations, duration, history,
	probabilityHistory, stuckCount
	``

print_matrix: Digunakan untuk mencetak matriks geladak sebagai tabel berbingkai, sama dengan genetic_algorithm.py.	```
	def print_matrix(matrix: dict) -> None:
	    cellWidth = max(2, len(str(max(matrix.values()))))
	    segment = "─" * (cellWidth + 2)
	    # Menggunakan unicode agar tabel menjadi rapih
	    print("┌" + "┬".join([segment] * shipHeight) + "┐")
	    for i in range(shipWidth):
	        cells = [f" {str(matrix[(i, j)]).rjust(cellWidth)}
	" for j in range(shipHeight)]
	        print("│" + "│".join(cells) + "│")
	        if i < shipWidth - 1:
	            print("├" + "┼".join([segment] * shipHeight) +
	"┤")
	    print("└" + "┴".join([segment] * shipHeight) + "┘")
	``

divider: Digunakan untuk mencetak garis pemisah dengan judul untuk memisahkan bagian output, sama dengan genetic_algorithm.py.	`def divider(title: str = "", width: int = 60, char: str = "=") -> None:`
	`    if title == "":`
	`        print(char * width)`
	`        return`
	`    label = f" {title} "`
	`    left = (width - len(label)) // 2`
	`    right = width - len(label) - left`
	`    print()`
	`    print(char * left + label + char * right)`
	`    print()`

build_table: Digunakan untuk membuat satu tabel teks berbingkai (judul, header, dan isi), sama dengan genetic_algorithm.py.	`def build_table(title: str, headers: tuple, rows: list) -> list:`
	`    codeW = max([len(headers[0])] + [len(str(c)) for c, _ in rows])`
	`    typeW = max([len(headers[1])] + [len(t) for _, t in rows])`
	`    totalW = codeW + typeW + 7`
	`    border = lambda l, m, r: l + "─" * (codeW + 2) + m + "─" * (typeW + 2) + r`
	`    lines = [`
	`        title.center(totalW),`
	`        border("┌", "┬", "┐"),`
	`        f"│ {headers[0].ljust(codeW)} │ {headers[1].ljust(typeW)} │",`
	`        border("├", "┼", "┤"),`
	`    ]`
	`    for c, t in rows:`
	`        lines.append(f"│ {str(c).rjust(codeW)} │ {t.ljust(typeW)} │")`
	`    if not rows:`
	`        lines.append(f"│ {'-'.center(codeW)} │ {'-'.center(typeW)} │")`
	`    lines.append(border("└", "┴", "┘"))`
	`    return lines`

print_vehicle_tables: Digunakan untuk mencetak tabel PLACED dan UNPLACED secara berdampingan, sama dengan genetic_algorithm.py.	`def print_vehicle_tables(placed: list, unplaced: list) -> None:`
	`    left = build_table("PLACED", ("Code", "Type"), placed)`
	`    right = build_table("UNPLACED", ("Jumlah", "Jenis"), unplaced)`
	`    height = max(len(left), len(right))`
	`    while len(left) < height:`
	`        left.append(" " * len(left[0]))`
	`    while len(right) < height:`
	`        right.append(" " * len(right[0]))`

	``
	    for a, b in zip(left, right):
	        print(a + " " + b)
	``

build_ship_matrix: Digunakan untuk mengubah state menjadi matriks geladak. Isi sel adalah nomor urut kendaraan di dalam kapal (sama dengan Code pada tabel PLACED) dan 0 untuk sel kosong, sama dengan hill_climbing.py.	```
	def build_ship_matrix(currentState: shipState) -> dict:
	    matrix = {(i, j): 0 for i in range(shipWidth) for j in range(shipHeight)}
	    number = 1
	    for vehicle in currentState:
	        if vehicle["position"] is None:
	            continue
	        for cell in get_cells(vehicle):
	            matrix[cell] = number
	        number += 1
	    return matrix
	``

build_vehicle_lists: Digunakan untuk membuat daftar kendaraan PLACED (Code, Type) dan UNPLACED (Jumlah, Jenis), sama dengan hill_climbing.py.	```
	def build_vehicle_lists(currentState: shipState):
	    placed = []
	    counter = {"mobil": 0, "minibus": 0, "truk": 0}
	    number = 1
	    for vehicle in currentState:
	        if vehicle["position"] is not None:
	            placed.append((number, vehicle["type"]))
	            number += 1
	        else:
	            counter[vehicle["type"]] += 1
	    unplaced = []
	    for jenis in counter:
	        if counter[jenis] > 0:
	            unplaced.append((f"{counter[jenis]}x", jenis))
	    return placed, unplaced
	``

print_state: Digunakan untuk menampilkan matriks geladak, tabel kendaraan di dalam dan di luar kapal, value, dan total berat sebuah state.	```
	def print_state(currentState: shipState) -> None:
	    placed, unplaced = build_vehicle_lists(currentState)
	    print_matrix(build_ship_matrix(currentState))
	    print_vehicle_tables(placed, unplaced)
	    print(f"Value : {calculate_value(currentState)}, total berat : {calculate_weight(currentState)}/{shipMaxCapacity} ton")
	``

plot_history_cli: Digunakan untuk menggambar grafik titik (iterasi, nilai) di CLI dengan format yang sama dengan plot_history dan plot_fitness_cli. Karena iterasi SA mencapai	```
	def plot_history_cli(points: list, title: str, decimals: int = 1, height: int = 12, plotWidth: int = 60) -> None:
	    print(title)
	    if not points:
	        print("(tidak ada data)")
	        return
	    maxX = max(x for x, _ in points)
	    low = min(y for _, y in points)
	``

ribuan, iterasi dikelompokkan menjadi 60 kolom.	
input_number: Digunakan untuk membaca parameter dari pengguna dengan validasi angka dan batas, serta memakai nilai default jika input kosong.	
main: Digunakan untuk membaca parameter, menjalankan SA sebanyak numberRuns kali, lalu menampilkan state awal, state akhir, ringkasan, dan grafik setiap	

```
    high = max(y for _, y in points)
    if high == low:
        high = low + 1
    grid = [[" "] * plotWidth for _ in range(height + 1)]
    for x, y in points:
        column = min(plotWidth - 1, x * plotWidth // (maxX
+ 1))
        row = round((y - low) / (high - low) * height)
        grid[height - row][column] = "*"
    for row in range(height + 1):
        value = low + (high - low) * (height - row) /
height
        print(f"{value:7.{decimals}f} |" +
"".join(grid[row]))
    print("        └" + "─" * plotWidth)
    # label sumbu x setiap 10 kolom
    labels = ""
    for column in range(0, plotWidth, 10):
        labels += str(column * (maxX + 1) //
plotWidth).ljust(10)
    print("         " + labels + str(maxX) + " (iterasi)")
```

```
def input_number(prompt: str, default, minimum,
maximum=None):
    while True:
        userInput = input(f"{prompt} (default {default}):
").strip()
        if userInput == "":
            return default
        try:
            value = type(default)(userInput)
            if value >= minimum and (maximum is None or
value <= maximum):
                return value
            if maximum is None:
                print(f"Masukkan angka minimal {minimum}.")
            else:
                print(f"Masukkan angka antara {minimum} dan
{maximum}.")
        except ValueError:
            print("Input harus berupa angka.")
```

```
def main():
    divider("SIMULATED ANNEALING")
    initialTemperature = input_number("Masukkan suhu awal",
100.0, 0.0001)
    coolingRate = input_number("Masukkan laju pendinginan
(0 < rate < 1)", 0.999, 0.0001, 0.9999)
```

run serta ringkasan eksperimen.	```
	    minTemperature = input_number("Masukkan suhu minimum", 0.01, 0.0001)
	    maxIteration = input_number("Masukkan batas iterasi", 20000, 1)
	    stuckThreshold = input_number("Masukkan batas iterasi tanpa perbaikan (stuck)", 200, 1)
	
	    results = []
	    for run in range(numberRuns):
	        divider(f"RUN {run + 1}")
	
	        print("Membuat state awal...")
	        initialState = create_initial_state(create_vehicles())
	
	        divider("STATE AWAL")
	        print_state(initialState)
	
	        print("Menjalankan simulated annealing...")
	        endState, iterations, duration, history, probabilityHistory, stuckCount = simulated_annealing(
	            initialState, initialTemperature, coolingRate, minTemperature, maxIteration, stuckThreshold)
	
	        divider("STATE AKHIR")
	        print_state(endState)
	
	        print("Ringkasan run ke-" + str(run + 1) + "...")
	        print(f"Value awal : {history[0]}")
	        print(f"Value akhir : {history[-1]}")
	        print(f"Banyak iterasi : {iterations}")
	        print(f"Durasi : {duration:.2f} detik")
	        print(f"Frekuensi stuck : {stuckCount} kali (tanpa perbaikan {stuckThreshold} iterasi)")
	        plot_history_cli(list(enumerate(history)), "Grafik value terhadap iterasi (* = value)")
	        print()
	        plot_history_cli(probabilityHistory, "Grafik e^(ΔE/T) terhadap iterasi (* = e^(ΔE/T), hanya saat ΔE < 0)", decimals=2)
	
	        results.append({
	            "initialState": initialState,
	            "endState": endState,
	            "startValue": history[0],
	            "endValue": history[-1],
	            "iterations": iterations,
	            "duration": duration,
	            "stuckCount": stuckCount,
	            "history": history,
	            "probabilityHistory": probabilityHistory,
	``

```
        })

    divider("RINGKASAN EKSPERIMEN")
    print(f"{'Run':<6}{'Value awal':>12}{'Value
akhir':>13}{'Iterasi':>9}{'Durasi (s)':>12}{'Stuck':>8}")
    for i in range(len(results)):
        r = results[i]
        print(f"{i +
1:<6}{r['startValue']:>12}{r['endValue']:>13}{r['iterations
']:>9}{r['duration']:>12.2f}{r['stuckCount']:>8}")
```

2.3.2 Hasil Algoritma

Eksperimen dijalankan dengan parameter default (initialTemperature = 100, coolingRate = 0,999, minTemperature = 0,01, maxIteration = 20.000, stuckThreshold = 200). Sesuai spesifikasi, SA dijalankan sebanyak 3 run (Run 1–3) yang dibahas secara rinci di bawah. Run 1–3 merupakan tiga run pertama dari eksekusi tersebut. Durasi hanya mengukur proses pencarian, tanpa waktu menampilkan output. Ringkasan hasil seluruh run adalah sebagai berikut:

(Ini adalah run eksperimen yang dijalankan sebanyak 10 run(s), untuk default run tetap berjumlah 3 sesuai spesifikasi)

Run	Value awal (juta)	Value akhir (juta)	Iterasi	Durasi (detik)	Frekuensi stuck
1	190	330	9205	0,55	43
2	210	340	9205	0,64	44
3	170	330	9205	0,58	43
4	190	340	9205	0,57	43
5	210	340	9205	0,63	43
6	180	330	9205	0,57	43
7	220	310	9205	0,49	44
8	200	340	9205	0,61	43
9	210	330	9205	0,57	41
10	230	340	9205	0,65	44

Didapatkan bahwa terdapat kedekatan dengan global optimum. Dengan batas berat 100 ton dan luas kapal 35 sel (7x5), value terbesar yang mungkin dicapai adalah 350 juta, yaitu 16 mobil dan 1 minibus (92 ton, mengisi tepat 35 sel). Kombinasi bernilai 340 juta ada beberapa seperti yang terlihat pada tabel di atas pada run 2,4,5,8, dan 10. Dengan rata-rata (dari 10 run) 333 juta, semua run berada pada rentang 40 juta dari global optimum. Global optimum jarang dicapai karena susunannya sangat sempit: seluruh 35 sel harus terisi tanpa celah. Untuk berpindah dari susunan bernilai 330-340 juta ke susunan tersebut, umumnya diperlukan beberapa langkah yang sementara menurunkan value (misalnya mengeluarkan satu kendaraan untuk menggeser kendaraan lain). Pada suhu rendah, langkah seperti itu hampir selalu ditolak, sehingga pencarian terjebak di local optimum.

Frekuensi stuck berada pada rentang 41-44 kali per run. Dengan 9.205 iterasi dan jendela 200 iterasi, frekuensi maksimum yang mungkin adalah 46 kali, sehingga hampir seluruh jendela tidak menghasilkan value terbaik yang baru. Hal ini menunjukkan bahwa value terbaik setiap run dicapai relatif awal, dan sisa pencarian (terutama fase suhu rendah setelah iterasi ke-3.000) hanya bergerak di sekitar local optimum tanpa perbaikan. Perlu dicatat bahwa dengan definisi ini frekuensi stuck sebanding dengan banyak iterasi, sehingga lebih tepat dibandingkan antar-run dengan jadwal pendinginan yang sama.

Berikut adalah hasil salah satu run yang akan muncul apabila menjalankan simulated_annealing.py. Untuk sisanya dapat dilihat pada GitHub pada bagian \output. Run 1. Run ini dimulai dari state awal bernilai 190 juta dan berakhir pada state akhir bernilai 330 juta setelah 9205 iterasi, dengan durasi pencarian 0,55 detik dan frekuensi stuck sebanyak 43 kali.

Running ke-1	State Awal

State awal
Value : 190
Total berat : 52 ton
Placed : mobil 8x, minibus 1x

Value : 190
Total berat : 52 ton
Placed : mobil 8x, minibus 1x

RINGKASAN EKSPERIMEN

Value awal :190
Value akhir : 330
Iterasi : 9205
Durasi (s) : 0.55
Stuck : 43

```
──────────────────────── RUN 1 ────────────────────────

Membuat state awal...

──────────────────────── STATE AWAL ───────────────────────
```

0	0	4	8	0	5	5
0	0	4	8	0	0	6
0	0	9	9	9	7	6
0	3	0	0	2	7	1
0	3	0	0	2	0	1

PLACED

Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	mobil
6	mobil
7	mobil
8	mobil
9	minibus

UNPLACED

Jumlah	Jenis
12x	mobil
6x	minibus
5x	truk

```
Value : 190, total berat : 52/100 ton
Menjalankan simulated annealing...
```

State Akhir

```
═══════════════════════ STATE AKHIR ═══════════════════════
```

13	13	13	9	9	11	11
15	15	15	14	8	4	2
1	10	10	14	8	4	2
1	0	7	14	6	6	5
12	12	7	3	3	0	5

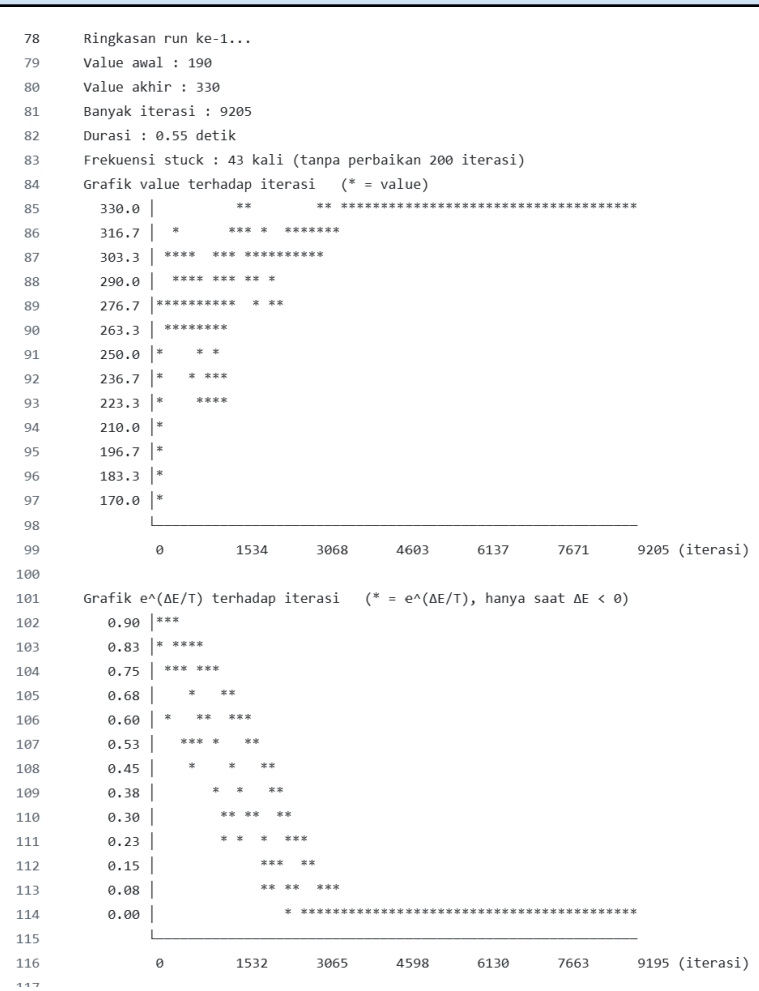
PLACED

Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	mobil
6	mobil
7	mobil
8	mobil
9	mobil
10	mobil
11	mobil
12	mobil
13	minibus
14	minibus
15	minibus

UNPLACED

Jumlah	Jenis
8x	mobil
4x	minibus
5x	truk

```
Value : 330, total berat : 96/100 ton
```

Ringkasan Run 1	
Ringkasan run, grafik value terhadap iterasi, dan grafik e^(ΔE/T) terhadap iterasi	

2.4 Penyelesaian dengan Genetic Algorithm

Algoritma Genetika adalah pencarian dan optimasi berbasis komputer yang meniru proses seleksi alam untuk menemukan solusi terbaik dari suatu masalah kompleks. Algoritma ini bekerja dengan cara membuat sekumpulan solusi acak (populasi), mengevaluasi kualitasnya (fitness), serta menduplikasi proses biologis seperti seleksi, persilangan (crossover), dan mutasi secara berulang hingga ditemukan solusi yang paling optimal.

Setelah beberapa konsiderasi bentuk genetik dalam solusi algoritma ini, dipilih bahwa representasi akan ditulis sebagai tipe data list dengan 17 tuple (jumlah kendaraan maksimum yang dapat diisi pada geladak kapal apabila menggunakan kendaraan terkecil yaitu mobil). Setiap tuple dari list akan diisi oleh angka tergantung tipe kendaraan, dengan 1 untuk mobil, 2 untuk minibus, 3 untuk truk dan 0 apabila slot kosong. Contoh langsung dari representasi genetiknya sebagai berikut:

genome dari genetik berikut *"[0, 0, 3, 3, 2, 2, 1, 1, 1, 0, 1, 1, 1, 2, 3, 2, 1]"* = dibaca sebagai *kosong, kosong, truk, truk, minibus, minibus, mobil, mobil, ... dan seterusnya*

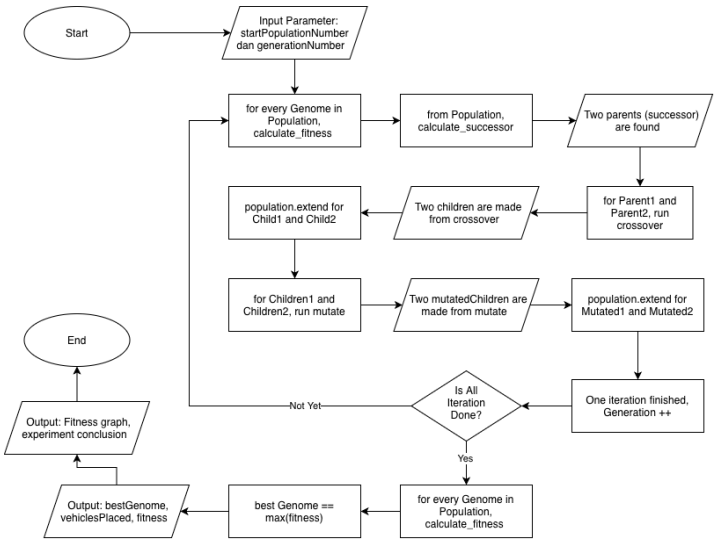
Representasi genetik ini akan diiterasi dengan fungsi yang menghitung kumulatif beban dan penempatannya pada matriks kapal, mulai dari digit pertama, hingga digit yang menyebabkan beban melebihi batas maksimum atau menyebabkan kapal penuh. Setelah melewati digit tersebut, apabila beban masih belum memenuhi maksimal, kapal dapat diisi dengan kendaraan selanjutnya di urutan list dengan berat yang menambah total beban dan memaksimalkan mendekati batas beban kapal (namun tetap dibawah maksimal kapal).

Perlu ditekankan bahwa bentuk tersebut akan menyebabkan gen tidak merepresentasikan kondisi kapal yang sebenarnya, karena yang akan masuk di dalam kapal terpotong oleh jumlah kumulatif beban atau space kosong, jadi beberapa gen memiliki sekian digit akhir yang sebenarnya tidak termasuk pada kapal. Digit-digit tersebut dipertahankan demi diversitas genetik, jika malah di replace menjadi 0, proses peranakan tidak akan mendapatkan hasil yang maksimal atau beragam. Adapun untuk perhitungan fitness function, hanya digunakan sampai cutoff point yang telah ditemukan saat perhitungan beban. Untuk memudahkan pemahaman, berikut contoh cara menghitung fitness function :

Genome		**[3, 2, 2, 1, 1, 1, 2, 1, 1, 2, 2, 0, 1, 3, 3, 1, 1]**			
Digit Ke-	**Jenis Kendaraan**	**Beban Kapal Kumulatif**	**Apakah Ada Space Kosong?**	**Masuk Kapal?**	**Fitness Function Kumulatif**
1	Truk	0+30	Ya (dari 35, terpakai 6)	Ya	0+60
2	Minibus	30+12	Ya (dari 35, terpakai 9)	Ya	60+30
3	Minibus	42+12	Ya (dari 35, terpakai 12)	Ya	90+30
4	Mobil	54+5	Ya (dari 35, terpakai 14)	Ya	120+20
5	Mobil	59+5	Ya (dari 35, terpakai 16)	Ya	140+20
6	Mobil	64+5	Ya (dari 35, terpakai 18)	Ya	160+20
7	Minibus	69+12	Ya (dari 35, terpakai 21)	Ya	180+30
8	Mobil	81+5	Ya (dari 35, terpakai 23)	Ya	210+20
9	Mobil	86+5	Ya (dari 35, terpakai 25)	Ya	230+20
10	Minibus	91+12	Ya (dari 35, terpakai 28)	Tidak, berat tidak cukup	-
11	Mobil	91+5	Ya (dari 35, terpakai 27)	Ya	250+20
	Kendaraan Lainnya	96++	Tergantung ukuran	Tidak akan muat secara beban	Final Fitness : 270

Jadi, dari genome tersebut diperoleh fitness number atau objective function 270, dengan daftar kendaraan masuk dalam kapal adalah [3221112111] setelah perhitungan dengan spesifikasi kapal. Mekanisme perhitungan ini sama untuk setiap genome dari genetik yang ada pada populasi, baik itu dari hasil peranakan/crossover, ataupun juga mutasi.

Pada program, proses pengolahan genetika algoritma dimulai dengan input parameter berupa populasi awal (startPopulationNumber) dan jumlah generasi (generationNumber) yang akan dihasilkan. Generasi ini dianggap sebagai iterasi. Ketika program menerima parameter, program akan menjalankan generationNumber iterasi dengan populasi awal sebanyak startPopulationNumber. Untuk setiap generasi, dua children akan dihasilkan dari crossover, beserta dua mutatedChildren dari mutation. Keempat genome hasil iterasi ini dimasukkan kepada populasi. Jadi, diperoleh rumus populasi akhir adalah = startPopulationNumber + 4*generationNumber. Untuk memudahkan pemahaman terhadap cara kerja genetik algoritma dalam permasalahan ini, berikut *flowchart*: drawio



2.4.1 Implementasi Algoritma

Hasil implementasi penuh algoritma dapat diakses pada GitHub, branch main atau azza-geneticalgorithm. Berikut adalah penjelasan dan cantuman source code dari seluruh fungsi yang ada pada algoritma.

Deskripsi	Source Code

generate_genetic: Digunakan untuk membuat satu genetik acak sepanjang 17 digit, diambil dari pool stok kendaraan (termasuk digit 0 sebagai slot kosong) tanpa melebihi stok.	`def generate_genetic(p: population) -> None:`
	`  pool = []`
	`  for code, amount in vehicleStock.items():`
	`  pool += [code] * amount`
	`  # Mengambil dari pool sebanyak genomeLength`
	`  individual = random.sample(pool, genomeLength)`
	`  print(f"Menghasilkan: {individual}")`
	`  p.append({"genome": individual, "fitness": 0})`

generate_population:
Digunakan untuk membuat populasi awal sebanyak jumlah yang diinput dengan memanggil generate_genetic berulang.
	`def generate_population(startPopulationNumber: int) -> population:`
	`  population = []`
	`  # Memanggil fungsi generate_genetic`
	`  for _ in range(startPopulationNumber):`
	`  generate_genetic(population)`
	`  return population`

check_space:
Digunakan untuk mencari koordinat pertama di kapal yang kosong dan cukup luas untuk satu kendaraan, lalu mengembalikan status muat dan posisinya.
	`def check_space(vehicle: dict):`
	`  vehicleWidth = vehicle["dimensions"][0]`
	`  vehicleHeight = vehicle["dimensions"][1]`
	`  positionX = 0;`
	`  positionY = 0;`
	`  canplace = False`
	`  for i in range(shipWidth):`
	`  for j in range(shipHeight):`
	`  # Jika koordinat terisi, skip`
	`  if shipMatrix[(i, j)] != 0:`
	`  continue`
	`  if i + vehicleWidth > shipWidth or j + vehicleHeight > shipHeight:`
	`  continue`
	`  canplace = True`
	`  for k in range(vehicleWidth):`
	`  for l in range(vehicleHeight):`
	`  if shipMatrix[(i + k, j + l)] != 0:`
	`  canplace = False`
	`  break`
	`  if not canplace:`
	`  break`
	`  if canplace:`
	`  positionX = i`
	`  positionY = j`
	`  return canplace, positionX, positionY`
	`  return canplace, positionX, positionY`

rotate_vehicle:
Digunakan untuk Mengembalikan salinan kendaraan dengan ukuran dan orientasi diputar, tanpa mengubah data kendaraan aslinya.
	`def rotate_vehicle(vehicle: dict) -> dict:`
	`  rotatedVehicle = vehicle.copy()`
	`  rotatedVehicle["dimensions"] = (vehicle["dimensions"][1], vehicle["dimensions"][0])`
	`  rotatedVehicle["orientation"] = "vertical" if vehicle["orientation"] == "horizontal" else "horizontal"`
	`  return rotatedVehicle`

place_vehicle: Digunakan untuk menandai sel-sel kapal yang ditempati	`def place_vehicle(vehicle: dict, positionX: int, positionY: int, vehicleCode: int) -> None:` `  vehicleWidth = vehicle["dimensions"][0]`

kendaraan dengan nomor urut kendaraan tersebut.	<pre>vehicleHeight = vehicle["dimensions"][1]
	for i in range(vehicleWidth):
	for j in range(vehicleHeight):
	shipMatrix[(positionX + i, positionY + j)] = vehicleCode</pre>

calculate_fitness:
Digunakan untuk menghitung fitness satu genetik dengan menempatkan kendaraan sesuai urutan genome selama muat dan tidak melebihi kapasitas berat, lalu menjumlahkan shipping fee. Juga mencatat kendaraan yang placed dan unplaced.
	<pre>def calculate_fitness(gen: dict):
	shipCurrWeight = 0
	shippingFee = 0
	numberVehiclePlaced = 0
	placedCount = {1: 0, 2: 0, 3: 0}
	gen["vehicles_placed"] = []
	# Mengiterasikan setiap digit dari genome
	for gene in gen["genome"]:
	if gene == 0:
	continue
	# Jika digit tidak 0, panggil fungsi check_space
	vehicle = vehicles[gene]
	canPlace, positionX, positionY = check_space(vehicle)
	# Kalau masih belum bisa di place, coba di rotate
	if not canPlace:
	vehicle = rotate_vehicle(vehicle)
	canPlace, positionX, positionY = check_space(vehicle)
	# Jika bisa dan weight dari ship masih memadai, panggil place_vehicle
	if canPlace and shipCurrWeight + vehicle["weight"] <= shipMaxCapacity:
	numberVehiclePlaced += 1
	placedCount[gene] += 1
	place_vehicle(vehicle, positionX, positionY, numberVehiclePlaced)
	gen["vehicles_placed"].append((numberVehiclePlaced, vehicle["id"]))
	shipCurrWeight += vehicle["weight"]
	shippingFee += vehicle["shippingFee"]
	# Menghitung vehicles yang tidak di place juga
	# Untuk mengetahui sisa stock kendaraan
	gen["vehicles_unplaced"] = []
	for code in (1, 2, 3):
	sisa = vehicleStock[code] - placedCount[code]
	if sisa > 0:
	gen["vehicles_unplaced"].append((f"{sisa}x", vehicles[code]["id"]))
	gen["fitness"] = shippingFee</pre>

reset_ship:	<pre>def reset_ship() -> None:</pre>

Digunakan untuk mengosongkan seluruh sel kapal sebelum menghitung individu berikutnya.	`global shipMatrix` `shipMatrix = {(i, j): 0 for i in range(shipWidth) for j in range(shipHeight)}`

print_matrix:
Digunakan untuk mencetak kondisi kapal sebagai tabel berbingkai yang menunjukkan posisi tiap kendaraan.
	`def print_matrix(matrix: dict) -> None:`
	`  cellWidth = max(2, len(str(max(matrix.values()))))`
	`  segment = "─" * (cellWidth + 2)`
	`  # Menggunakan unicode agar tabel menjadi rapih`
	`  print("┌" + "┬".join([segment] * shipHeight) + "┐")`
	`  for i in range(shipWidth):`
	`  cells = [f" {str(matrix[(i, j)]).rjust(cellWidth)} " for j in range(shipHeight)]`
	`  print("│" + "│".join(cells) + "│")`
	`  if i < shipWidth - 1:`
	`  print("├" + "┼".join([segment] * shipHeight) + "┤")`
	`  print("└" + "┴".join([segment] * shipHeight) + "┘")`

divider:
Digunakan untuk mencetak garis pemisah dengan judul untuk memisahkan tiap tahap proses.
	`def divider(title: str = "", width: int = 60, char: str = "=") -> None:`
	`  if title == "":`
	`  print(char * width)`
	`  return`
	`  label = f" {title} "`
	`  left = (width - len(label)) // 2`
	`  right = width - len(label) - left`
	`  print()`
	`  print(char * left + label + char * right)`
	`  print()`

build_table:
Digunakan untuk membuat satu tabel teks berbingkai (judul, header, baris) dan mengembalikannya sebagai daftar baris.
	`def build_table(title: str, headers: tuple, rows: list) -> list:`
	`  codeW = max([len(headers[0])] + [len(str(c)) for c, _ in rows])`
	`  typeW = max([len(headers[1])] + [len(t) for _, t in rows])`
	`  totalW = codeW + typeW + 7`
	`  border = lambda l, m, r: l + "─" * (codeW + 2) + m + "─" * (typeW + 2) + r`
	`  lines = [`
	`  title.center(totalW),`
	`  border("┌", "┬", "┐"),`
	`  f"│ {headers[0].ljust(codeW)} │ {headers[1].ljust(typeW)} │",`
	`  border("├", "┼", "┤"),`
	`  ]`
	`  for c, t in rows:`
	`  lines.append(f"│ {str(c).rjust(codeW)} │`

	{t.ljust(typeW)} |") if not rows: lines.append(f"| {'-'.center(codeW)} | {'-'.center(typeW)} |") lines.append(border("└", "┴", "┘")) return lines

print_vehicle_tables:
Digunakan untuk mencetak tabel kendaraan placed dan unplaced berdampingan.
	def print_vehicle_tables(placed: list, unplaced: list) -> None:
	left = build_table("PLACED", ("Code", "Type"), placed)
	right = build_table("UNPLACED", ("Jumlah", "Jenis"),
	unplaced)
	height = max(len(left), len(right))
	while len(left) < height:
	left.append(" " * len(left[0]))
	while len(right) < height:
	right.append(" " * len(right[0]))
	for a, b in zip(left, right):
	print(a + " " + b)

plot_fitness_cli:
Digunakan untuk menggambar grafik teks fitness maksimum dan rata-rata terhadap banyak iterasi.
	def plot_fitness_cli(maxHistory: list, avgHistory: list,
	height: int = 12) -> None:
	allValues = maxHistory + avgHistory
	low = min(allValues)
	high = max(allValues)
	if high == low:
	high = low + 1
	columnGap = 6 if len(maxHistory) <= 12 else 4
	width = len(maxHistory) * columnGap
	
	# * = maksimum, o = rata-rata (maksimum menimpa rata-rata
	kalau sama)
	grid = [[" "] * width for _ in range(height + 1)]
	for idx in range(len(maxHistory)):
	x = idx * columnGap + 2
	yMax = round((maxHistory[idx] - low) / (high - low) *
	height)
	yAvg = round((avgHistory[idx] - low) / (high - low) *
	height)
	grid[height - yAvg][x] = "o"
	grid[height - yMax][x] = "*"
	print("Grafik fitness terhadap iterasi (* = maksimum, o =
	rata-rata)")
	for row in range(height + 1):
	value = high - (high - low) * row / height
	print(f"{value:7.1f} |" + "".join(grid[row]))
	print(" └" + "─" * width)
	labels = "".join(str(i).ljust(columnGap) for i in

	range(len(maxHistory))) print(" " + labels + " (iterasi)")

calculate_successor:
Digunakan untuk memilih dua parent dari populasi berdasarkan peluang sebanding fitness (roulette), dengan memastikan kedua parent punya genome berbeda.
	def calculate_successor(population: population) -> list:
	totalFitness = 0
	for gen in population:
	totalFitness += gen["fitness"]
	# chance adalah probabilitas genome terpilih sebagai suksesor
	for gen in population:
	chance = gen["fitness"] / totalFitness if totalFitness > 0 else 0
	gen["chance"] = chance
	parent1 = random.choices(population, weights=[gen["chance"] for gen in population], k=1)[0]
	# mencari kandidat parent 2 yang tidak sama dengan parent 1
	candidateParent2 = [gen for gen in population if gen["genome"] != parent1["genome"]]
	if len(candidateParent2) == 0:
	return [parent1, parent1]
	chanceParent2 = [gen["chance"] for gen in candidateParent2]
	if sum(chanceParent2) == 0:
	parent2 = random.choice(candidateParent2)
	else:
	parent2 = random.choices(candidateParent2, weights=chanceParent2, k=1)[0]
	return [parent1, parent2]

build_child:
Digunakan untuk membentuk satu anak dari potongan dua parent pada titik potong tertentu, lalu memperbaiki digit yang melebihi stok kendaraan.
	def build_child(first: list, second: list, p: int) -> list:
	genome = first[:p] + second[p:]
	# Mengecek stock dari vehicle dari genome
	leftovers = vehicleStock.copy()
	for i in range(genomeLength):
	g = genome[i]
	if leftovers[g] > 0:
	leftovers[g] -= 1
	else:
	# Jika stok jenis suatu habis, maka
	# ganti dengan jenis yang masih tersisa pada stock
	leftoverDigit = [c for c in leftovers if leftovers[c] > 0]
	newDigit = random.choice(leftoverDigit)
	genome[i] = newDigit
	leftovers[newDigit] -= 1
	return genome

crossover:	def crossover(parentA: dict, parentB: dict):

Digunakan untuk menghasilkan dua anak dari dua parent dengan titik potong acak, memanggil build_child untuk masing-masing.	`p = random.randint(1, genomeLength - 1)`
	`genomeA = build_child(parentA["genome"], parentB["genome"], p)`
	`genomeB = build_child(parentB["genome"], parentA["genome"], p)`
	`return {"genome": genomeA, "fitness": 0}, {"genome": genomeB, "fitness": 0}`

mutate:
Digunakna untuk memutasi salinan genetik dengan peluang tertentu per digit, lewat tukar posisi atau ganti digit, tanpa melanggar stok kendaraan.
	`def mutate(child: dict, rate: float) -> dict:`
	`genome = child["genome"].copy()`
	`leftovers = vehicleStock.copy()`
	`for g in genome:`
	`leftovers[g] -= 1`
	`for i in range(genomeLength):`
	`if random.random() < rate:`
	`if random.random() < 0.5:`
	`j = random.randint(0, genomeLength - 1)`
	`genome[i], genome[j] = genome[j], genome[i]`
	`else:`
	`# Memastikan digit baru mutasi memenuhi stok vehicle`
	`newDigit = random.randint(0, 3)`
	`if leftovers[newDigit] > 0:`
	`leftovers[genome[i]] += 1`
	`leftovers[newDigit] -= 1`
	`genome[i] = newDigit`
	`return {"genome": genome, "fitness": 0}`

input_integer:
Digunakan untuk meminta input angka bulat dari user dan mengulang pertanyaan sampai nilainya valid dan memenuhi batas minimum.
	`def input_integer(prompt: str, minimum: int) -> int:`
	`while True:`
	`try:`
	`value = int(input(prompt))`
	`if value >= minimum:`
	`return value`
	`print(f"Masukkan angka minimal {minimum}.")`
	`except ValueError:`
	`print("Input harus berupa angka bulat.")`

main:
Digunakan untuk menjalankan seluruh alur: input parameter, populasi awal, iterasi (parent, crossover, mutasi), pemilihan genetik terbaik, grafik, dan ringkasan eksperimen.
	`def main():`
	`divider("GENETIC ALGORITHM")`
	`startPopulationNumber = input_integer("Masukkan jumlah populasi awal: ", 2) # Minimum 2 populasi awal`
	`generationNumber = input_integer("Masukkan jumlah generasi: ", 1)`
	`divider("POPULASI AWAL")`
	`print(f"Menghasilkan populasi awal sebanyak " + str(startPopulationNumber)+" genetik...")`

```
    population = generate_population(startPopulationNumber)
    startTime = perf_counter()
    delayAtStart = delayTime

    divider("KONDISI KAPAL AWAL")
    print(f"Mengecek kondisi awal kapal...")
    print(f"Kondisi kapal ukuran {shipHeight} x {shipWidth}
dengan kapasitas maksimal {shipMaxCapacity} ton:")
    print_matrix(shipMatrix)

    divider("FITNESS FUNCTION POPULASI")
    print(f"Menghitung fitness function untuk tiap individu pada
populasi...")
    for gen in population:
        calculate_fitness(gen)
        print(f"Genetik: {gen['genome']}, Fitness:
{gen['fitness']}")
        reset_ship()
    maxHistory = [max(gen["fitness"] for gen in population)]
    avgHistory = [sum(gen["fitness"] for gen in population) /
len(population)]

    divider("ITERASI GENERASI")
    print(f"Memulai iterasi untuk " + str(generationNumber) + "
generasi...")
    for generation in range(generationNumber):
        divider(f"GENERASI {generation + 1}")
        divider("PARENT")
        print(f"Memilih dua individu sesuai chance untuk menjadi
parent...")
        successors = calculate_successor(population)

print(f"PARENT1:{successors[0]['genome']}\nPARENT2:{successors[
1]['genome']}")
        divider("CROSSOVER")
        print(f"Menjalankan fungsi crossover pada parent...")
        children = crossover(successors[0], successors[1])
        for child in children:
            calculate_fitness(child)
            reset_ship()
        print(f"Memperoleh 2 children hasil crossover kedua
parent")

print(f"CHILD1:{children[0]['genome']}\nCHILD2:{children[1]['ge
nome']}")
        population.extend(children)
```

```
        print(f"Menambahkan children ke dalam populasi...")
        print(f"Berhasil menambahkan children ke dalam populasi,
total populasi: {len(population)}")
        divider("MUTATION")
        print(f"Menjalankan fungsi mutate pada children...")
        mutatedChildren = [mutate(child, 0.1) for child in
children]
        for mutant in mutatedChildren:
            calculate_fitness(mutant)
            reset_ship()
        print(f"Memperoleh 2 mutated children hasil mutasi kedua
child:")
print(f"MUTATED1:{mutatedChildren[0]['genome']}\nMUTATED2:{muta
tedChildren[1]['genome']}")
        population.extend(mutatedChildren)
        print(f"Menambahkan mutated children ke dalam
populasi...")
        print(f"Berhasil menambahkan mutated ke dalam populasi,
total populasi: {len(population)}")

        # Mencatat fitness maksimum dan rata-rata di akhir
iterasi
        maxHistory.append(max(gen["fitness"] for gen in
population))
        avgHistory.append(sum(gen["fitness"] for gen in
population) / len(population))
    divider("ITERASI SELESAI")
    print(f"Iterasi untuk {generationNumber} generasi selesai,
populasi akhir: {len(population)}")
    print(f"Menghitung fitness function untuk tiap individu pada
populasi akhir")
    for gen in population:
        calculate_fitness(gen)
        print(f"Genetik: {gen['genome']}, Fitness:
{gen['fitness']}")
        reset_ship()
    best = max(population, key=lambda gen: gen["fitness"])
    totalTime = perf_counter() - startTime
    algorithmTime = totalTime - (delayTime - delayAtStart)

    divider("BEST GENETIC")
    print(f"Fitness function tertinggi pada populasi akhir:
{best['fitness']}")
    print(f"Diraih oleh individu dengan genetik:
{best['genome']}")
```

```
    calculate_fitness(best)
    print_vehicle_tables(best["vehicles_placed"],
best["vehicles_unplaced"])
    print(f"Matriks kapal setelah penempatan kendaraan:")
    print_matrix(shipMatrix)
    reset_ship()

    divider("GRAFIK FITNESS")
    plot_fitness_cli(maxHistory, avgHistory)
    print()
    for i in range(len(maxHistory)):
        print(f"Iterasi {i}: maks = {maxHistory[i]}, rata-rata =
{avgHistory[i]:.1f}")
    divider("RINGKASAN EKSPERIMEN")
    print(f"Jumlah populasi awal  : {startPopulationNumber}")
    print(f"Banyak iterasi        : {generationNumber}")
    print(f"Fitness akhir         : {best['fitness']}")
    print(f"Durasi total          : {totalTime:.2f} detik
(termasuk delay tampilan)")
    print(f"Durasi algoritma      : {algorithmTime:.4f} detik
(tanpa delay tampilan)")
```

2.4.2 Hasil Algoritma

Hasil algoritma genetik dijalankan sebanyak total 18 kali, dengan rincian berupa sembilan kali untuk populasi sebagai kontrol, dan sembilan kali untuk generasi/iterasi sebagai kontrol. Masing-masing variabel kontrol dipasangkan dengan tiga variabel independent dan dijalankan tiga kali untuk melihat konektivitas antara variabel kontrol dan independent. Secara lengkap, informasi output yang akan dianalisis dapat diakses melalui Repository GitHub. Karena iterasi mencakup sangat banyak pengulangan, dan ratusan line yang menunjukkan proses pencarian, pembahasan hanya akan memasukkan cuplikan atau deskripsi singkat hasil, state awal dan state akhir dari setiap iterasi.

2.4.2.1 Populasi sebagai Variabel Kontrol

<table>
<tr><th colspan="2">Untuk Populasi = 5, Iterasi/Generasi = 5x</th></tr>
<tr><td>Running ke-1</td><td>State Awal</td></tr>
<tr><td rowspan="2">Pada running kode pertama dengan Populasi = 5, Iterasi/Generasi = 5x, diperoleh informasi berikut:

Iterasi 0: maks = 300, rata-rata = 272.0
Iterasi 1: maks = 300, rata-rata = 270.0
Iterasi 2: maks = 300, rata-rata = 272.3
Iterasi 3: maks = 300, rata-rata = 274.1
Iterasi 4: maks = 310, rata-rata = 273.3
Iterasi 5: maks = 310, rata-rata = 268.4

═══════════
RINGKASAN EKSPERIMEN
═══════════

Jumlah populasi awal : 5
Banyak iterasi : 5
Fitness akhir : 310
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0086 detik (tanpa delay tampilan)</td><td>
<pre>
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 5 genetik...
Menghasilkan: [1, 1, 0, 2, 0, 1, 3, 0, 1, 3, 2, 1, 3, 0, 1, 1, 1]
Menghasilkan: [2, 1, 0, 1, 1, 1, 2, 2, 0, 1, 1, 0, 1, 1, 2, 3, 1]
Menghasilkan: [1, 0, 1, 3, 1, 2, 2, 1, 1, 1, 2, 1, 3, 0, 2, 3, 0]
Menghasilkan: [1, 3, 1, 0, 1, 2, 1, 1, 0, 2, 0, 0, 0, 0, 0, 2, 3]
Menghasilkan: [1, 0, 2, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 2, 1, 2, 0]

═══════════ KONDISI KAPAL AWAL ═══════════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
</pre>

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
</td></tr>
<tr><td>State Akhir</td></tr>
<tr><td>
<pre>
═══════════ BEST GENETIC ═══════════

Fitness function tertinggi pada populasi akhir: 310
Diraih oleh individu dengan genetik: [0, 1, 0, 1, 1, 1, 2, 2, 2, 1, 2, 0, 1, 1, 2, 3, 1]
</pre>

PLACED

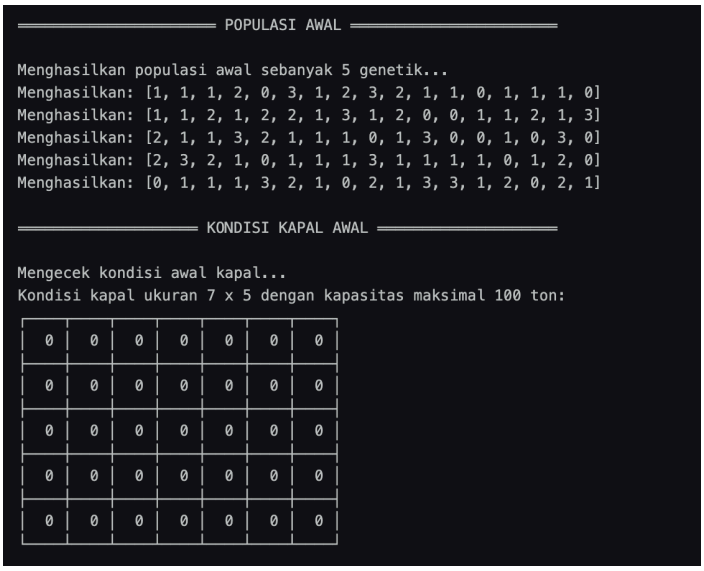
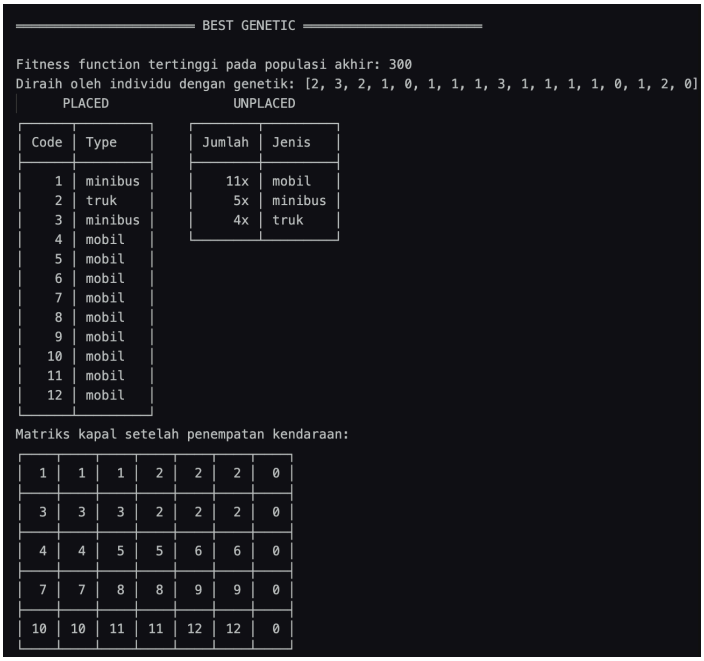
Code	Type
1	mobil
2	mobil
3	mobil
4	mobil
5	minibus
6	minibus
7	minibus
8	mobil
9	minibus
10	mobil
11	mobil
12	minibus
13	mobil

UNPLACED

Jumlah	Jenis
12x	mobil
2x	minibus
5x	truk

<pre>
Matriks kapal setelah penempatan kendaraan:
</pre>

1	1	2	2	3	3	0
4	4	5	5	5	8	8
6	6	6	7	7	7	0
9	9	9	10	10	11	11
12	12	12	13	13	0	0
</td></tr>
</table>

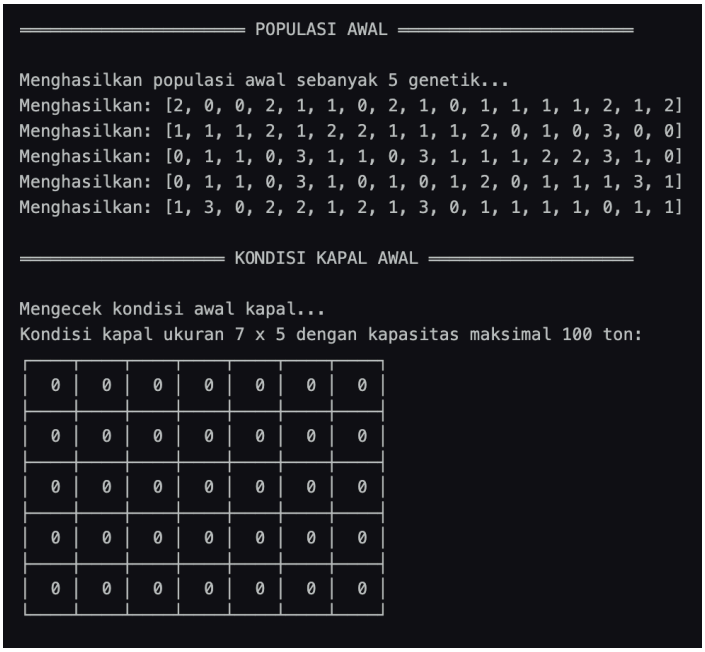
Running ke-2	State Awal
Pada running kode kedua dengan Populasi = 5, Iterasi/Generasi = 5x, diperoleh informasi berikut: Iterasi 0: maks = 300, rata-rata = 274.0 Iterasi 1: maks = 300, rata-rata = 265.6 Iterasi 2: maks = 300, rata-rata = 263.8 Iterasi 3: maks = 300, rata-rata = 264.7 Iterasi 4: maks = 300, rata-rata = 265.2 Iterasi 5: maks = 300, rata-rata = 261.2	
RINGKASAN EKSPERIMEN Jumlah populasi awal : 5 Banyak iterasi : 5 Fitness akhir : 300 Durasi total : 0.01 detik (termasuk delay tampilan) Durasi algoritma : 0.0074 detik (tanpa delay tampilan)	State Akhir 
Running ke-3	State Awal

Pada running kode ketiga dengan Populasi = 5, Iterasi/Generasi = 5x, diperoleh informasi berikut:

Iterasi 0: maks = 310, rata-rata = 282.0
Iterasi 1: maks = 310, rata-rata = 283.3
Iterasi 2: maks = 310, rata-rata = 277.7
Iterasi 3: maks = 320, rata-rata = 277.6
Iterasi 4: maks = 320, rata-rata = 273.3
Iterasi 5: maks = 320, rata-rata = 273.6

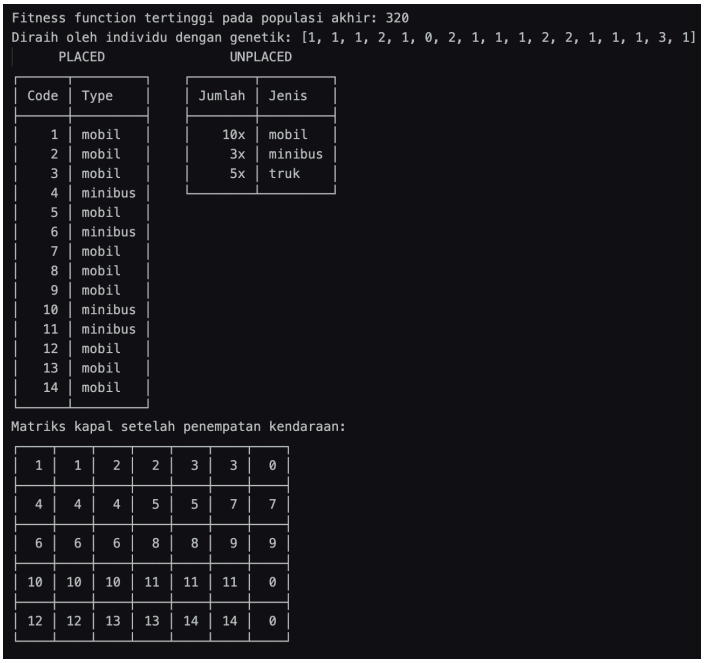
RINGKASAN EKSPERIMEN

Jumlah populasi awal : 5
Banyak iterasi : 5
Fitness akhir : 320
Durasi total : 0.00 detik (termasuk delay tampilan)
Durasi algoritma : 0.0038 detik (tanpa delay tampilan)



0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

State Akhir



Code	Type
1	mobil
2	mobil
3	mobil
4	minibus
5	mobil
6	minibus
7	mobil
8	mobil
9	mobil
10	minibus
11	minibus
12	mobil
13	mobil
14	mobil

Jumlah	Jenis
10x	mobil
3x	minibus
5x	truk

1	1	2	2	3	3	0
4	4	4	5	5	7	7
6	6	6	8	8	9	9
10	10	10	11	11	11	0
12	12	13	13	14	14	0

Untuk Populasi = 10, Iterasi/Generasi = 10x

Running ke-1	State Awal

Pada running kode pertama dengan Populasi = 10, Iterasi/Generasi = 10x, diperoleh informasi berikut:

Iterasi 0: maks = 290, rata-rata = 280.0
Iterasi 1: maks = 310, rata-rata = 282.2
Iterasi 2: maks = 310, rata-rata = 280.8
Iterasi 3: maks = 310, rata-rata = 277.1
Iterasi 4: maks = 310, rata-rata = 274.8
Iterasi 5: maks = 310, rata-rata = 274.0
Iterasi 6: maks = 310, rata-rata = 274.8
Iterasi 7: maks = 310, rata-rata = 273.0
Iterasi 8: maks = 310, rata-rata = 274.3
Iterasi 9: maks = 310, rata-rata = 273.7
Iterasi 10: maks = 310, rata-rata = 274.0

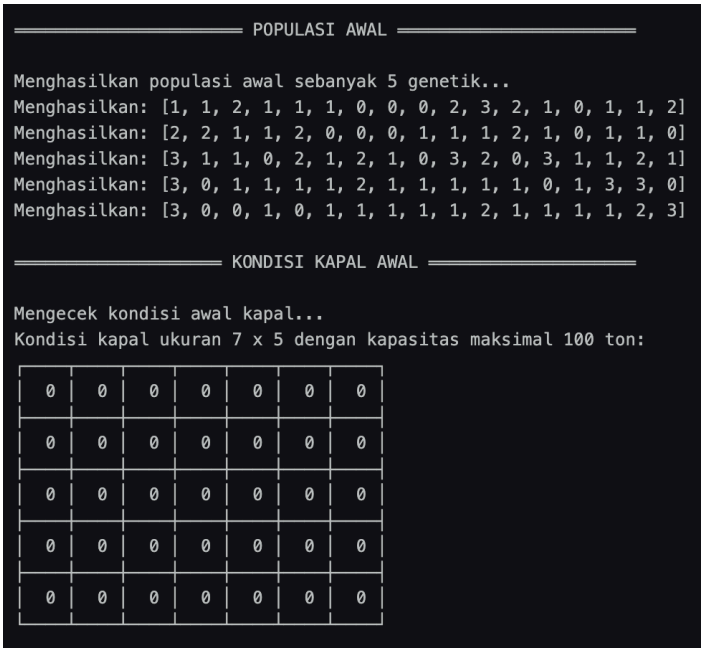
═══════════════

RINGKASAN EKSPERIMEN

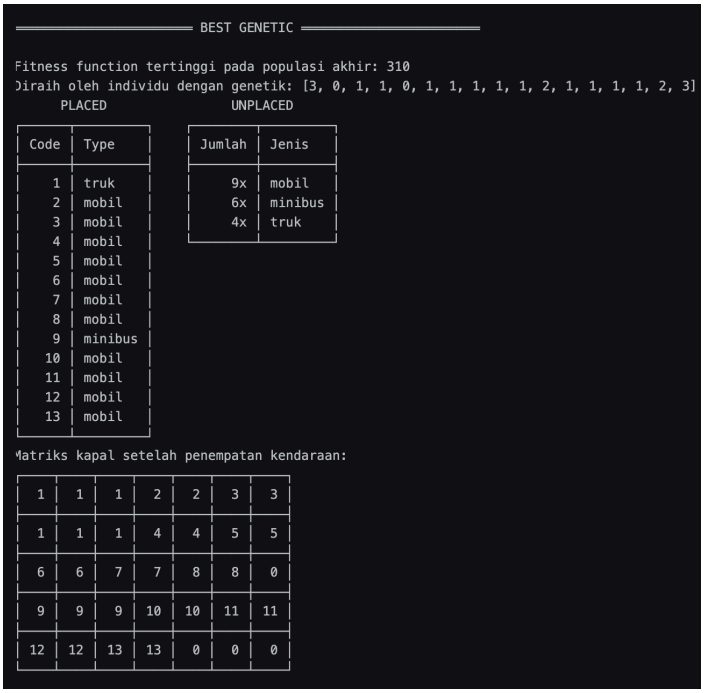
═══════════════

Jumlah populasi awal : 5
Banyak iterasi : 10
Fitness akhir : 310
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0179 detik (tanpa delay tampilan)

Running ke-2



State Akhir



State Awal

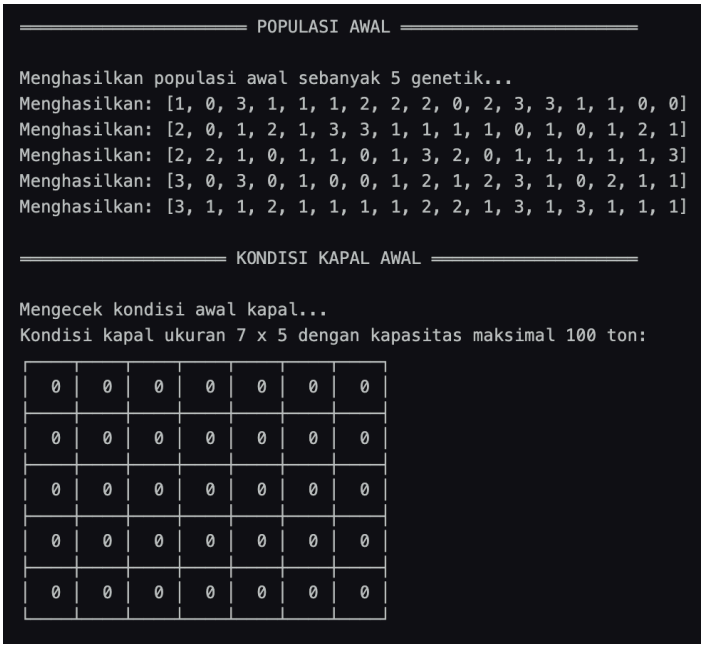
Pada running kode kedua dengan Populasi = 10, Iterasi/Generasi = 10x, diperoleh informasi berikut:

Iterasi 0: maks = 270, rata-rata = 256.0
Iterasi 1: maks = 270, rata-rata = 257.8
Iterasi 2: maks = 270, rata-rata = 256.9
Iterasi 3: maks = 270, rata-rata = 252.9
Iterasi 4: maks = 270, rata-rata = 251.9
Iterasi 5: maks = 300, rata-rata = 255.6
Iterasi 6: maks = 300, rata-rata = 255.5
Iterasi 7: maks = 300, rata-rata = 254.8
Iterasi 8: maks = 300, rata-rata = 255.4
Iterasi 9: maks = 300, rata-rata = 254.9
Iterasi 10: maks = 300, rata-rata = 255.3

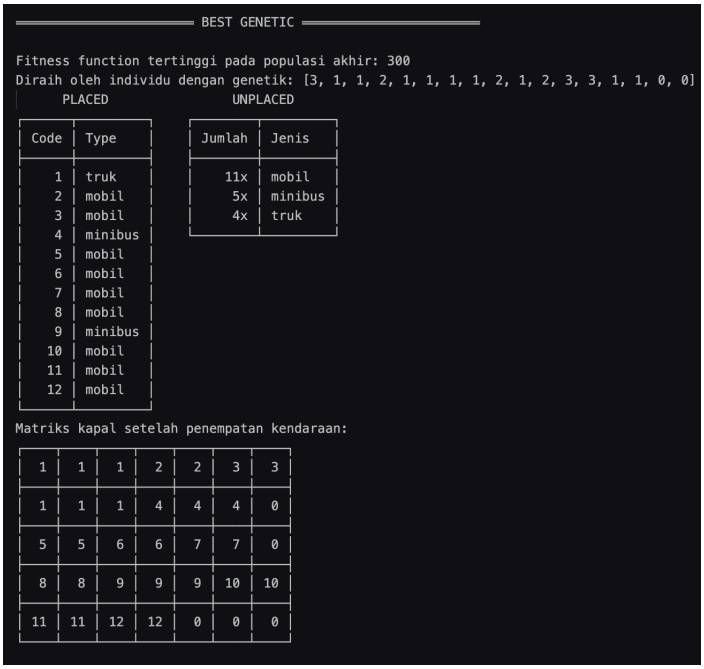
RINGKASAN EKSPERIMEN

Jumlah populasi awal : 5
Banyak iterasi : 10
Fitness akhir : 300
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0158 detik (tanpa delay tampilan)

Running ke-3



State Akhir



State Awal

Pada running kode ketiga dengan Populasi = 10, Iterasi/Generasi = 10x, diperoleh informasi berikut:

Iterasi 0: maks = 280, rata-rata = 258.0
Iterasi 1: maks = 280, rata-rata = 258.9
Iterasi 2: maks = 280, rata-rata = 256.2
Iterasi 3: maks = 280, rata-rata = 254.7
Iterasi 4: maks = 280, rata-rata = 254.8
Iterasi 5: maks = 280, rata-rata = 254.8
Iterasi 6: maks = 280, rata-rata = 254.1
Iterasi 7: maks = 280, rata-rata = 255.5
Iterasi 8: maks = 300, rata-rata = 257.0
Iterasi 9: maks = 310, rata-rata = 258.3
Iterasi 10: maks = 310, rata-rata = 258.2

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 5
Banyak iterasi : 10
Fitness akhir : 310
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0158 detik (tanpa delay tampilan)

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 5 genetik...
Menghasilkan: [2, 1, 1, 1, 0, 3, 1, 2, 1, 0, 1, 2, 3, 3, 2, 2, 1]
Menghasilkan: [2, 1, 3, 0, 1, 3, 0, 2, 0, 1, 0, 2, 0, 2, 3, 0, 2]
Menghasilkan: [1, 3, 3, 1, 1, 3, 2, 1, 1, 0, 1, 1, 3, 2, 2, 2, 2]
Menghasilkan: [1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 3, 2, 2, 2, 2, 2, 1]
Menghasilkan: [0, 3, 1, 1, 1, 1, 3, 3, 1, 0, 2, 0, 0, 1, 1, 1, 2]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 310
Diraih oleh individu dengan genetik: [2, 1, 1, 1, 2, 1, 1, 2, 1, 0, 1, 2, 3, 1, 0, 2, 1]
      PLACED                UNPLACED
┌──────┬─────────┐     ┌────────┬─────────┐
│ Code │ Type    │     │ Jumlah │ Jenis   │
├──────┼─────────┤     ├────────┼─────────┤
│    1 │ minibus │     │    12x │ mobil   │
│    2 │ mobil   │     │     2x │ minibus │
│    3 │ mobil   │     │     5x │ truk    │
│    4 │ mobil   │     └────────┴─────────┘
│    5 │ minibus │
│    6 │ mobil   │
│    7 │ mobil   │
│    8 │ minibus │
│    9 │ mobil   │
│   10 │ mobil   │
│   11 │ minibus │
│   12 │ mobil   │
│   13 │ minibus │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  1 │  2 │  2 │  3 │  3 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  5 │  5 │  5 │  6 │  6 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  8 │  8 │  8 │  9 │  9 │
├────┼────┼────┼────┼────┼────┼────┤
│ 10 │ 10 │ 11 │ 11 │ 11 │ 12 │ 12 │
├────┼────┼────┼────┼────┼────┼────┤
│ 13 │ 13 │ 13 │  0 │  0 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

Untuk Populasi = 10, Iterasi/Generasi = 15x

Running ke-1	State Awal

Pada running kode pertama dengan Populasi = 10, Iterasi/Generasi = 15x, diperoleh informasi berikut:

Iterasi 0: maks = 300, rata-rata = 270.0
Iterasi 1: maks = 300, rata-rata = 266.7
Iterasi 2: maks = 300, rata-rata = 264.6
Iterasi 3: maks = 300, rata-rata = 269.4
Iterasi 4: maks = 300, rata-rata = 268.6
Iterasi 5: maks = 300, rata-rata = 266.8
Iterasi 6: maks = 300, rata-rata = 267.9
Iterasi 7: maks = 300, rata-rata = 271.8
Iterasi 8: maks = 300, rata-rata = 270.0
Iterasi 9: maks = 300, rata-rata = 269.5
Iterasi 10: maks = 320, rata-rata = 270.4
Iterasi 11: maks = 320, rata-rata = 270.2
Iterasi 12: maks = 320, rata-rata = 268.9
Iterasi 13: maks = 320, rata-rata = 269.6
Iterasi 14: maks = 320, rata-rata = 271.0
Iterasi 15: maks = 320, rata-rata = 271.7

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 5
Banyak iterasi : 15
Fitness akhir : 320
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0134 detik (tanpa delay tampilan)

Running ke-2

```
═══════════════════ POPULASI AWAL ═══════════════════

Menghasilkan populasi awal sebanyak 5 genetik...
Menghasilkan: [3, 1, 0, 2, 0, 3, 1, 1, 3, 1, 2, 0, 1, 2, 1, 1, 3]
Menghasilkan: [1, 1, 3, 3, 1, 1, 0, 0, 1, 1, 0, 1, 3, 1, 0, 2, 0]
Menghasilkan: [3, 1, 1, 1, 0, 1, 1, 1, 1, 2, 2, 0, 1, 2, 1, 1, 1]
Menghasilkan: [1, 1, 3, 1, 2, 1, 1, 3, 1, 2, 1, 0, 2, 1, 1, 1, 3]
Menghasilkan: [2, 1, 1, 2, 3, 1, 2, 0, 0, 1, 3, 1, 1, 0, 2, 2, 0]

═══════════════ KONDISI KAPAL AWAL ═══════════════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════════════ BEST GENETIC ═══════════════

Fitness function tertinggi pada populasi akhir: 320
Diraih oleh individu dengan genetik: [2, 1, 1, 1, 0, 1, 1, 1, 1, 2, 2, 0, 1, 2, 1, 1, 3]
      PLACED                UNPLACED
┌──────┬─────────┐     ┌────────┬─────────┐
│ Code │ Type    │     │ Jumlah │ Jenis   │
├──────┼─────────┤     ├────────┼─────────┤
│    1 │ minibus │     │    10x │ mobil   │
│    2 │ mobil   │     │     3x │ minibus │
│    3 │ mobil   │     │     5x │ truk    │
│    4 │ mobil   │     └────────┴─────────┘
│    5 │ mobil   │
│    6 │ mobil   │
│    7 │ mobil   │
│    8 │ mobil   │
│    9 │ minibus │
│   10 │ minibus │
│   11 │ mobil   │
│   12 │ minibus │
│   13 │ mobil   │
│   14 │ mobil   │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  1 │  2 │  2 │  3 │  3 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  5 │  5 │  6 │  6 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  8 │  8 │  9 │  9 │  9 │
├────┼────┼────┼────┼────┼────┼────┤
│ 10 │ 10 │ 10 │ 11 │ 11 │ 13 │ 13 │
├────┼────┼────┼────┼────┼────┼────┤
│ 12 │ 12 │ 12 │ 14 │ 14 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

State Awal

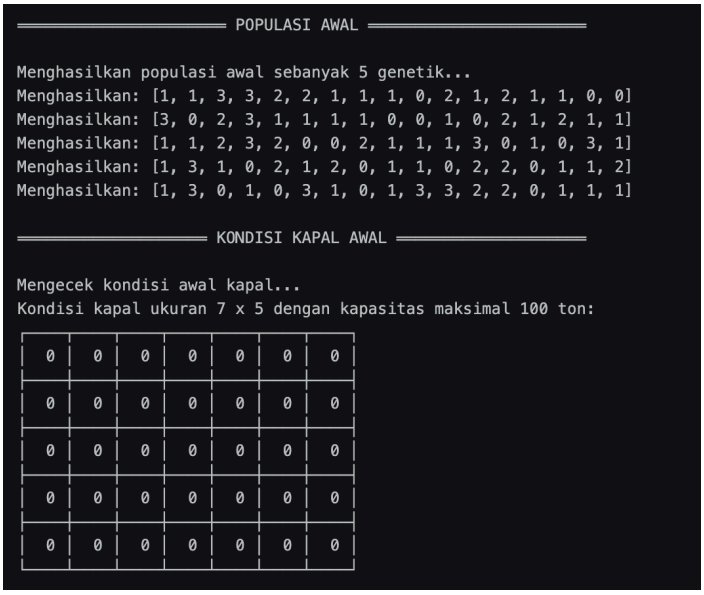
Pada running kode kedua dengan Populasi = 10, Iterasi/Generasi = 15x, diperoleh informasi berikut:

Iterasi 0: maks = 270, rata-rata = 256.0
Iterasi 1: maks = 270, rata-rata = 254.4
Iterasi 2: maks = 270, rata-rata = 252.3
Iterasi 3: maks = 270, rata-rata = 248.8
Iterasi 4: maks = 270, rata-rata = 247.1
Iterasi 5: maks = 280, rata-rata = 248.0
Iterasi 6: maks = 280, rata-rata = 246.9
Iterasi 7: maks = 280, rata-rata = 246.4
Iterasi 8: maks = 280, rata-rata = 245.9
Iterasi 9: maks = 280, rata-rata = 243.7
Iterasi 10: maks = 280, rata-rata = 243.8
Iterasi 11: maks = 280, rata-rata = 243.5
Iterasi 12: maks = 280, rata-rata = 244.7
Iterasi 13: maks = 280, rata-rata = 244.2
Iterasi 14: maks = 280, rata-rata = 243.9
Iterasi 15: maks = 280, rata-rata = 243.2

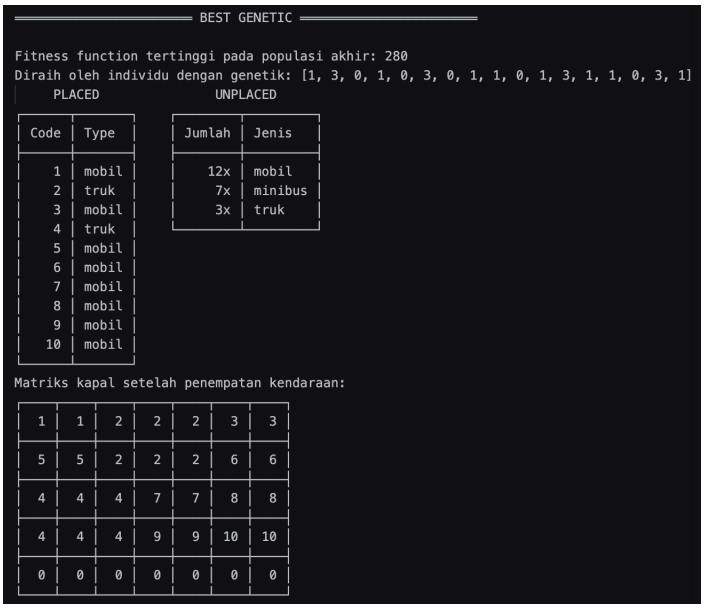
═══════════════════════════
RINGKASAN EKSPERIMEN
═══════════════════════════

Jumlah populasi awal : 5
Banyak iterasi : 15
Fitness akhir : 280
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0191 detik (tanpa delay tampilan)

Running ke-3



State Akhir



State Awal

Pada running kode ketiga dengan Populasi = 10, Iterasi/Generasi = 15x, diperoleh informasi berikut:

Iterasi 0: maks = 300, rata-rata = 270.0
Iterasi 1: maks = 300, rata-rata = 283.3
Iterasi 2: maks = 300, rata-rata = 285.4
Iterasi 3: maks = 320, rata-rata = 288.8
Iterasi 4: maks = 320, rata-rata = 292.9
Iterasi 5: maks = 320, rata-rata = 294.0
Iterasi 6: maks = 320, rata-rata = 291.4
Iterasi 7: maks = 320, rata-rata = 293.6
Iterasi 8: maks = 320, rata-rata = 292.2
Iterasi 9: maks = 320, rata-rata = 293.4
Iterasi 10: maks = 320, rata-rata = 292.2
Iterasi 11: maks = 320, rata-rata = 290.0
Iterasi 12: maks = 320, rata-rata = 290.4
Iterasi 13: maks = 320, rata-rata = 288.8
Iterasi 14: maks = 320, rata-rata = 289.8
Iterasi 15: maks = 320, rata-rata = 290.6

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 5
Banyak iterasi : 15
Fitness akhir : 320
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0121 detik (tanpa delay tampilan)

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 5 genetik...
Menghasilkan: [1, 1, 0, 0, 2, 1, 1, 1, 1, 1, 0, 2, 1, 1, 2, 0, 1]
Menghasilkan: [3, 3, 1, 1, 1, 1, 1, 0, 2, 3, 0, 1, 3, 1, 1, 2, 1]
Menghasilkan: [1, 2, 1, 1, 1, 2, 1, 1, 1, 1, 0, 3, 3, 1, 2, 0, 1]
Menghasilkan: [2, 0, 1, 1, 2, 1, 0, 1, 1, 2, 3, 1, 0, 1, 0, 1, 1]
Menghasilkan: [3, 1, 0, 2, 2, 1, 0, 3, 1, 0, 1, 1, 1, 1, 1, 1, 0]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 320
Diraih oleh individu dengan genetik: [1, 2, 1, 1, 1, 2, 1, 1, 1, 1, 0, 2, 0, 1, 2, 0, 1]
      PLACED                UNPLACED
┌──────┬─────────┐     ┌────────┬────────┐
│ Code │ Type    │     │ Jumlah │ Jenis  │
├──────┼─────────┤     ├────────┼────────┤
│    1 │ mobil   │     │    10x │ mobil  │
│    2 │ minibus │     │     3x │ minibus│
│    3 │ mobil   │     │     5x │ truk   │
│    4 │ mobil   │     └────────┴────────┘
│    5 │ mobil   │
│    6 │ minibus │
│    7 │ mobil   │
│    8 │ mobil   │
│    9 │ mobil   │
│   10 │ mobil   │
│   11 │ minibus │
│   12 │ mobil   │
│   13 │ minibus │
│   14 │ mobil   │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  2 │  2 │  2 │  3 │  3 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  5 │  5 │  6 │  6 │  6 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  8 │  8 │  9 │  9 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│ 10 │ 10 │ 11 │ 11 │ 11 │ 12 │ 12 │
├────┼────┼────┼────┼────┼────┼────┤
│ 13 │ 13 │ 13 │ 14 │ 14 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

2.3.2.2 Generasi sebagai Variabel Kontrol

<table>
<tr><th colspan="2">Untuk Iterasi/Generasi = 10, Populasi = 3</th></tr>
<tr><td>Running ke-1</td><td>State Awal</td></tr>
<tr><td rowspan="2">Pada running kode pertama dengan Iterasi/Generasi = 10, Populasi = 3, diperoleh informasi berikut:

Iterasi 0: maks = 300, rata-rata = 276.7
Iterasi 1: maks = 300, rata-rata = 278.6
Iterasi 2: maks = 300, rata-rata = 280.9
Iterasi 3: maks = 300, rata-rata = 278.0
Iterasi 4: maks = 300, rata-rata = 276.3
Iterasi 5: maks = 310, rata-rata = 278.7
Iterasi 6: maks = 310, rata-rata = 280.0
Iterasi 7: maks = 310, rata-rata = 279.7
Iterasi 8: maks = 310, rata-rata = 276.9
Iterasi 9: maks = 310, rata-rata = 275.6
Iterasi 10: maks = 310, rata-rata = 275.6

═══════════════════════
RINGKASAN EKSPERIMEN
═══════════════════════

Jumlah populasi awal : 3
Banyak iterasi : 10
Fitness akhir : 310
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0139 detik (tanpa delay tampilan)</td><td>
<pre>
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 3 genetik...
Menghasilkan: [0, 2, 1, 1, 1, 1, 0, 2, 1, 1, 3, 3, 2, 1, 2, 0, 0]
Menghasilkan: [0, 1, 2, 1, 3, 1, 1, 1, 1, 1, 3, 1, 3, 1, 2, 1, 0]
Menghasilkan: [1, 3, 1, 2, 2, 1, 0, 2, 2, 2, 1, 0, 0, 1, 1, 1, 2]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
</pre>

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
</td></tr>
<tr><td>State Akhir</td></tr>
<tr><td>
<pre>
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 310
Diraih oleh individu dengan genetik: [0, 1, 2, 1, 3, 1, 1, 1, 1, 1, 3, 1, 1, 1, 2, 1, 0]
</pre>

PLACED

Code	Type
1	mobil
2	minibus
3	mobil
4	truk
5	mobil
6	mobil
7	mobil
8	mobil
9	mobil
10	mobil
11	mobil
12	mobil
13	mobil

UNPLACED

Jumlah	Jenis
9x	mobil
6x	minibus
4x	truk

<pre>
Matriks kapal setelah penempatan kendaraan:
</pre>

1	1	2	2	2	3	3
4	4	4	5	5	6	6
4	4	4	7	7	8	8
9	9	10	10	11	11	0
12	12	13	13	0	0	0
</td></tr>
</table>

Running ke-2	State Awal

Pada running kode kedua dengan Iterasi/Generasi = 10, Populasi = 3, diperoleh informasi berikut:

Iterasi 0: maks = 280, rata-rata = 256.7
Iterasi 1: maks = 300, rata-rata = 267.1
Iterasi 2: maks = 300, rata-rata = 264.5
Iterasi 3: maks = 300, rata-rata = 260.7
Iterasi 4: maks = 300, rata-rata = 257.9
Iterasi 5: maks = 300, rata-rata = 260.9
Iterasi 6: maks = 300, rata-rata = 257.0
Iterasi 7: maks = 300, rata-rata = 256.1
Iterasi 8: maks = 300, rata-rata = 255.1
Iterasi 9: maks = 300, rata-rata = 253.8
Iterasi 10: maks = 300, rata-rata = 254.4

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 3
Banyak iterasi : 10
Fitness akhir : 300
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0090 detik (tanpa delay tampilan)

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 3 genetik...
Menghasilkan: [0, 0, 3, 3, 2, 3, 3, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1]
Menghasilkan: [2, 1, 2, 1, 3, 1, 1, 0, 1, 0, 1, 3, 1, 2, 0, 1, 3]
Menghasilkan: [3, 1, 2, 0, 1, 1, 3, 1, 3, 0, 1, 2, 2, 2, 0, 1, 1]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
```

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

State Akhir

```
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 300
Diraih oleh individu dengan genetik: [2, 1, 2, 1, 3, 1, 1, 0, 1, 0, 1, 3, 1, 2, 0, 1, 1]
```

PLACED

Code	Type
1	minibus
2	mobil
3	minibus
4	mobil
5	truk
6	mobil
7	mobil
8	mobil
9	mobil
10	mobil
11	mobil
12	mobil

UNPLACED

Jumlah	Jenis
11x	mobil
5x	minibus
4x	truk

```
Matriks kapal setelah penempatan kendaraan:
```

1	1	1	2	2	4	4
3	3	3	5	5	5	0
6	6	0	5	5	5	0
7	7	8	8	9	9	0
10	10	11	11	12	12	0

Running ke-3	State Awal

Pada running kode ketiga dengan Iterasi/Generasi = 10, Populasi = 3, diperoleh informasi berikut:

Iterasi 0: maks = 270, rata-rata = 260.0
Iterasi 1: maks = 280, rata-rata = 268.6
Iterasi 2: maks = 310, rata-rata = 274.5
Iterasi 3: maks = 310, rata-rata = 270.0
Iterasi 4: maks = 310, rata-rata = 273.2
Iterasi 5: maks = 310, rata-rata = 274.3
Iterasi 6: maks = 310, rata-rata = 276.7
Iterasi 7: maks = 310, rata-rata = 276.1
Iterasi 8: maks = 310, rata-rata = 277.4
Iterasi 9: maks = 310, rata-rata = 274.6
Iterasi 10: maks = 310, rata-rata = 275.3

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 3
Banyak iterasi : 10
Fitness akhir : 310
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0078 detik (tanpa delay tampilan)

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 3 genetik...
Menghasilkan: [1, 0, 2, 2, 0, 0, 3, 3, 1, 3, 1, 1, 0, 0, 0, 1, 0]
Menghasilkan: [0, 1, 3, 2, 1, 1, 2, 0, 1, 1, 0, 2, 0, 1, 1, 3, 2]
Menghasilkan: [0, 1, 1, 1, 0, 1, 0, 2, 1, 2, 0, 2, 1, 3, 1, 2, 0]

═══════════ KONDISI KAPAL AWAL ═══════════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════════ BEST GENETIC ═══════════

Fitness function tertinggi pada populasi akhir: 310
Diraih oleh individu dengan genetik: [1, 1, 1, 2, 0, 1, 0, 2, 1, 2, 0, 2, 1, 1, 1, 3, 2]
      PLACED                UNPLACED

┌──────┬─────────┐     ┌────────┬─────────┐
│ Code │ Type    │     │ Jumlah │ Jenis   │
├──────┼─────────┤     ├────────┼─────────┤
│    1 │ mobil   │     │    12x │ mobil   │
│    2 │ mobil   │     │     2x │ minibus │
│    3 │ mobil   │     │     5x │ truk    │
│    4 │ minibus │     └────────┴─────────┘
│    5 │ mobil   │
│    6 │ minibus │
│    7 │ mobil   │
│    8 │ minibus │
│    9 │ minibus │
│   10 │ mobil   │
│   11 │ mobil   │
│   12 │ mobil   │
│   13 │ minibus │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  2 │  2 │  3 │  3 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  4 │  5 │  5 │  7 │  7 │
├────┼────┼────┼────┼────┼────┼────┤
│  6 │  6 │  6 │  8 │  8 │  8 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│  9 │  9 │  9 │ 10 │ 10 │ 11 │ 11 │
├────┼────┼────┼────┼────┼────┼────┤
│ 12 │ 12 │ 13 │ 13 │ 13 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

Iterasi/Generasi = 10, Populasi = 6

Running ke-1	State Awal

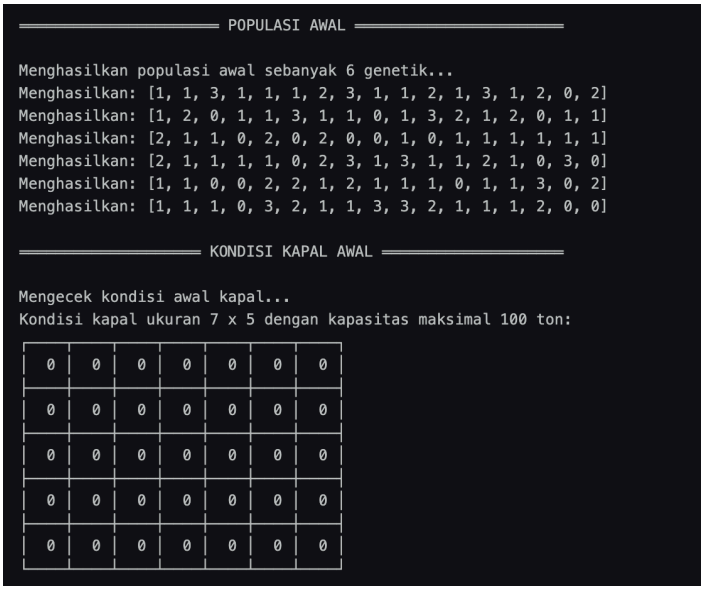
Pada running kode pertama dengan Iterasi/Generasi = 10, Populasi = 6, diperoleh informasi berikut:

Iterasi 0: maks = 300, rata-rata = 271.7
Iterasi 1: maks = 300, rata-rata = 270.0
Iterasi 2: maks = 300, rata-rata = 270.7
Iterasi 3: maks = 300, rata-rata = 270.0
Iterasi 4: maks = 300, rata-rata = 272.3
Iterasi 5: maks = 300, rata-rata = 271.9
Iterasi 6: maks = 300, rata-rata = 273.0
Iterasi 7: maks = 300, rata-rata = 272.6
Iterasi 8: maks = 300, rata-rata = 272.4
Iterasi 9: maks = 300, rata-rata = 272.6
Iterasi 10: maks = 300, rata-rata = 271.3

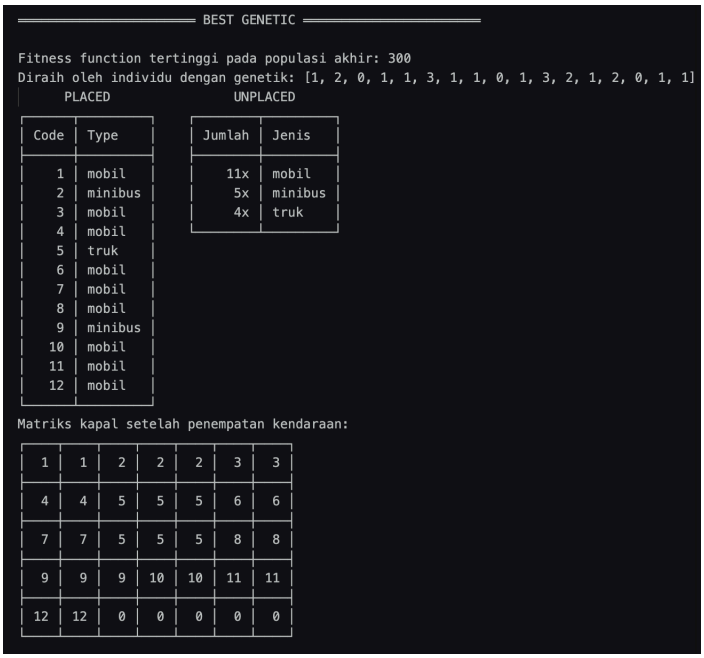
═══════════════════
RINGKASAN EKSPERIMEN
═══════════════════

Jumlah populasi awal : 6
Banyak iterasi : 10
Fitness akhir : 300
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0082 detik (tanpa delay tampilan)

Running ke-2



State Akhir



State Awal

Pada running kode kedua dengan Iterasi/Generasi = 10, Populasi = 6, diperoleh informasi berikut:

Iterasi 0: maks = 310, rata-rata = 283.3
Iterasi 1: maks = 310, rata-rata = 276.0
Iterasi 2: maks = 310, rata-rata = 280.7
Iterasi 3: maks = 310, rata-rata = 273.9
Iterasi 4: maks = 310, rata-rata = 275.0
Iterasi 5: maks = 310, rata-rata = 276.5
Iterasi 6: maks = 310, rata-rata = 275.0
Iterasi 7: maks = 310, rata-rata = 275.0
Iterasi 8: maks = 310, rata-rata = 276.6
Iterasi 9: maks = 310, rata-rata = 277.4
Iterasi 10: maks = 310, rata-rata = 277.0

═══════════════
RINGKASAN EKSPERIMEN
═══════════════

Jumlah populasi awal : 6
Banyak iterasi : 10
Fitness akhir : 310
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0124 detik (tanpa delay tampilan)

Running ke-3

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 6 genetik...
Menghasilkan: [3, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 2, 3, 3, 2]
Menghasilkan: [1, 1, 1, 1, 1, 0, 1, 0, 1, 2, 0, 0, 1, 2, 1, 1, 3]
Menghasilkan: [2, 3, 1, 2, 1, 0, 0, 2, 0, 1, 1, 2, 3, 0, 3, 1, 1]
Menghasilkan: [1, 1, 1, 0, 2, 1, 1, 1, 1, 1, 1, 3, 1, 1, 3, 0, 1]
Menghasilkan: [2, 0, 0, 1, 1, 2, 1, 1, 1, 1, 0, 1, 2, 1, 0, 1, 0]
Menghasilkan: [0, 0, 3, 2, 1, 2, 1, 1, 1, 3, 1, 1, 1, 1, 0, 1, 3]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 310
Diraih oleh individu dengan genetik: [1, 1, 1, 0, 2, 1, 1, 1, 1, 1, 1, 3, 1, 1, 3, 0, 1]
      PLACED                UNPLACED
┌──────┬─────────┐     ┌────────┬────────┐
│ Code │ Type    │     │ Jumlah │ Jenis  │
├──────┼─────────┤     ├────────┼────────┤
│    1 │ mobil   │     │     9x │ mobil  │
│    2 │ mobil   │     │     6x │ minibus│
│    3 │ mobil   │     │     4x │ truk   │
│    4 │ minibus │     └────────┴────────┘
│    5 │ mobil   │
│    6 │ mobil   │
│    7 │ mobil   │
│    8 │ mobil   │
│    9 │ mobil   │
│   10 │ mobil   │
│   11 │ truk    │
│   12 │ mobil   │
│   13 │ mobil   │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  2 │  2 │  3 │  3 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  4 │  5 │  5 │  6 │  6 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  8 │  8 │  9 │  9 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│ 10 │ 10 │ 11 │ 11 │ 11 │ 12 │ 12 │
├────┼────┼────┼────┼────┼────┼────┤
│ 13 │ 13 │ 11 │ 11 │ 11 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

State Awal

Pada running kode ketiga dengan Iterasi/Generasi = 10, Populasi = 6, diperoleh informasi berikut:

Iterasi 0: maks = 320, rata-rata = 276.7
Iterasi 1: maks = 320, rata-rata = 287.0
Iterasi 2: maks = 320, rata-rata = 279.3
Iterasi 3: maks = 320, rata-rata = 280.6
Iterasi 4: maks = 320, rata-rata = 278.2
Iterasi 5: maks = 320, rata-rata = 276.2
Iterasi 6: maks = 320, rata-rata = 277.0
Iterasi 7: maks = 320, rata-rata = 278.5
Iterasi 8: maks = 320, rata-rata = 276.8
Iterasi 9: maks = 320, rata-rata = 277.6
Iterasi 10: maks = 330, rata-rata = 278.7

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 6
Banyak iterasi : 10
Fitness akhir : 330
Durasi total : 0.01 detik (termasuk delay tampilan)
Durasi algoritma : 0.0124 detik (tanpa delay tampilan)

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 6 genetik...
Menghasilkan: [0, 1, 2, 0, 3, 0, 0, 0, 1, 1, 1, 1, 1, 1, 3, 1, 0]
Menghasilkan: [1, 0, 1, 3, 1, 1, 1, 1, 3, 2, 1, 0, 1, 1, 0, 3, 1]
Menghasilkan: [2, 2, 0, 0, 3, 3, 1, 3, 0, 0, 1, 0, 1, 1, 1, 2, 1]
Menghasilkan: [1, 1, 1, 2, 1, 0, 1, 3, 2, 2, 0, 1, 1, 1, 1, 1, 1]
Menghasilkan: [1, 1, 1, 0, 0, 1, 3, 1, 1, 0, 0, 1, 2, 1, 1, 2, 1]
Menghasilkan: [1, 2, 1, 2, 0, 1, 0, 1, 1, 2, 1, 1, 2, 3, 1, 1, 1]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 330
Diraih oleh individu dengan genetik: [1, 2, 1, 2, 0, 1, 0, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1]
      PLACED                UNPLACED
┌──────┬─────────┐     ┌────────┬────────┐
│ Code │ Type    │     │ Jumlah │ Jenis  │
├──────┼─────────┤     ├────────┼────────┤
│    1 │ mobil   │     │     8x │ mobil  │
│    2 │ minibus │     │     4x │ minibus│
│    3 │ mobil   │     │     5x │ truk   │
│    4 │ minibus │     └────────┴────────┘
│    5 │ mobil   │
│    6 │ mobil   │
│    7 │ mobil   │
│    8 │ minibus │
│    9 │ mobil   │
│   10 │ mobil   │
│   11 │ mobil   │
│   12 │ mobil   │
│   13 │ mobil   │
│   14 │ mobil   │
│   15 │ mobil   │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  2 │  2 │  2 │  3 │  3 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  4 │  5 │  5 │  6 │  6 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  8 │  8 │  8 │  9 │  9 │
├────┼────┼────┼────┼────┼────┼────┤
│ 10 │ 10 │ 11 │ 11 │ 12 │ 12 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│ 13 │ 13 │ 14 │ 14 │ 15 │ 15 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

Iterasi/Generasi = 10, Populasi = 12

Running ke-1	State Awal

Pada running kode pertama dengan Iterasi/Generasi = 10, Populasi = 12, diperoleh informasi berikut:

Iterasi 0: maks = 300, rata-rata = 262.5
Iterasi 1: maks = 300, rata-rata = 268.8
Iterasi 2: maks = 300, rata-rata = 269.0
Iterasi 3: maks = 300, rata-rata = 268.3
Iterasi 4: maks = 300, rata-rata = 266.4
Iterasi 5: maks = 300, rata-rata = 268.8
Iterasi 6: maks = 320, rata-rata = 272.8
Iterasi 7: maks = 320, rata-rata = 272.5
Iterasi 8: maks = 320, rata-rata = 274.5
Iterasi 9: maks = 320, rata-rata = 272.9
Iterasi 10: maks = 320, rata-rata = 272.7

═══════════════
RINGKASAN EKSPERIMEN
═══════════════

Jumlah populasi awal : 12
Banyak iterasi : 10
Fitness akhir : 320
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0160 detik (tanpa delay tampilan)

Running ke-2

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 12 genetik...
Menghasilkan: [1, 2, 1, 0, 0, 1, 0, 1, 0, 1, 2, 1, 3, 1, 2, 1, 3]
Menghasilkan: [1, 0, 3, 2, 2, 3, 1, 2, 2, 0, 1, 1, 1, 0, 1, 1, 3]
Menghasilkan: [2, 3, 1, 1, 1, 3, 2, 2, 1, 1, 0, 1, 1, 1, 2, 1, 1]
Menghasilkan: [1, 2, 2, 0, 2, 0, 1, 1, 0, 3, 0, 2, 1, 1, 1, 0, 1]
Menghasilkan: [3, 2, 3, 2, 2, 0, 0, 0, 1, 3, 1, 1, 1, 3, 0, 1, 1]
Menghasilkan: [2, 1, 3, 1, 1, 2, 1, 0, 1, 0, 0, 1, 3, 1, 1, 3, 1]
Menghasilkan: [1, 1, 1, 1, 0, 0, 1, 2, 1, 1, 2, 0, 1, 1, 0, 2, 3]
Menghasilkan: [3, 1, 2, 1, 1, 0, 1, 1, 2, 2, 1, 2, 1, 0, 3, 1, 1]
Menghasilkan: [1, 2, 1, 1, 2, 0, 0, 1, 0, 0, 3, 1, 1, 1, 1, 1, 1]
Menghasilkan: [1, 2, 0, 0, 1, 1, 3, 0, 2, 1, 2, 1, 3, 3, 2, 1, 2]
Menghasilkan: [2, 2, 0, 2, 0, 1, 3, 1, 1, 2, 1, 2, 1, 1, 3, 2, 1]
Menghasilkan: [1, 0, 3, 0, 1, 1, 2, 1, 3, 0, 0, 2, 0, 2, 1, 1, 1]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌───┬───┬───┬───┬───┬───┬───┐
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
├───┼───┼───┼───┼───┼───┼───┤
│ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │ 0 │
└───┴───┴───┴───┴───┴───┴───┘
```

State Akhir

```
═══════ BEST GENETIC ═══════

Fitness function tertinggi pada populasi akhir: 320
Diraih oleh individu dengan genetik: [1, 2, 1, 0, 0, 0, 1, 1, 2, 2, 1, 2, 1, 1, 1, 1, 1]
      PLACED                UNPLACED
┌──────┬──────────┐     ┌────────┬─────────┐
│ Code │ Type     │     │ Jumlah │ Jenis   │
├──────┼──────────┤     ├────────┼─────────┤
│    1 │ mobil    │     │    10x │ mobil   │
│    2 │ minibus  │     │     3x │ minibus │
│    3 │ mobil    │     │     5x │ truk    │
│    4 │ mobil    │     └────────┴─────────┘
│    5 │ mobil    │
│    6 │ minibus  │
│    7 │ minibus  │
│    8 │ mobil    │
│    9 │ minibus  │
│   10 │ mobil    │
│   11 │ mobil    │
│   12 │ mobil    │
│   13 │ mobil    │
│   14 │ mobil    │
└──────┴──────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  2 │  2 │  2 │  3 │  3 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  5 │  5 │  6 │  6 │  6 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  7 │  8 │  8 │ 10 │ 10 │
├────┼────┼────┼────┼────┼────┼────┤
│  9 │  9 │  9 │ 11 │ 11 │ 12 │ 12 │
├────┼────┼────┼────┼────┼────┼────┤
│ 13 │ 13 │ 14 │ 14 │  0 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

State Awal

Pada running kode kedua dengan Iterasi/Generasi = 10, Populasi = 12, diperoleh informasi berikut:

Iterasi 0: maks = 290, rata-rata = 270.0
Iterasi 1: maks = 290, rata-rata = 266.2
Iterasi 2: maks = 290, rata-rata = 259.5
Iterasi 3: maks = 290, rata-rata = 260.4
Iterasi 4: maks = 290, rata-rata = 261.1
Iterasi 5: maks = 300, rata-rata = 263.1
Iterasi 6: maks = 300, rata-rata = 262.5
Iterasi 7: maks = 300, rata-rata = 261.2
Iterasi 8: maks = 300, rata-rata = 262.3
Iterasi 9: maks = 300, rata-rata = 261.9
Iterasi 10: maks = 300, rata-rata = 262.5

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 12
Banyak iterasi : 10
Fitness akhir : 300
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0162 detik (tanpa delay tampilan)

Running ke-3

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 12 genetik...
Menghasilkan: [3, 3, 0, 3, 1, 1, 2, 1, 2, 0, 1, 2, 1, 1, 3, 0, 1]
Menghasilkan: [1, 1, 1, 0, 1, 3, 0, 3, 1, 1, 1, 2, 1, 1, 1, 0, 3]
Menghasilkan: [1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 2, 1, 0, 2, 2]
Menghasilkan: [1, 3, 1, 1, 3, 1, 1, 3, 1, 2, 1, 2, 0, 1, 3, 0, 0]
Menghasilkan: [1, 0, 1, 2, 1, 1, 1, 2, 3, 0, 0, 1, 1, 3, 0, 1, 2]
Menghasilkan: [1, 1, 3, 1, 1, 1, 0, 2, 1, 3, 1, 2, 1, 0, 2, 0, 2]
Menghasilkan: [0, 1, 1, 1, 1, 1, 1, 3, 2, 1, 1, 1, 0, 1, 0, 2, 2]
Menghasilkan: [1, 1, 1, 2, 0, 1, 3, 1, 3, 1, 1, 0, 0, 0, 2, 1, 1]
Menghasilkan: [0, 1, 3, 2, 1, 2, 0, 1, 2, 1, 0, 0, 1, 1, 3, 2, 1]
Menghasilkan: [3, 0, 3, 1, 1, 2, 1, 0, 2, 1, 1, 0, 1, 3, 0, 3, 0]
Menghasilkan: [1, 1, 1, 0, 1, 1, 3, 0, 3, 1, 1, 3, 1, 2, 2, 1, 2]
Menghasilkan: [1, 1, 1, 0, 1, 3, 2, 2, 1, 3, 1, 1, 0, 0, 1, 2, 3]

═══════════ KONDISI KAPAL AWAL ═══════════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
└─────┴─────┴─────┴─────┴─────┴─────┴─────┘
```

State Akhir

```
═══════════ BEST GENETIC ═══════════

Fitness function tertinggi pada populasi akhir: 300
Diraih oleh individu dengan genetik: [1, 1, 1, 0, 0, 1, 3, 1, 0, 1, 0, 2, 1, 3, 1, 2, 1]
      PLACED                UNPLACED
┌──────┬─────────┐     ┌────────┬─────────┐
│ Code │ Type    │     │ Jumlah │ Jenis   │
├──────┼─────────┤     ├────────┼─────────┤
│    1 │ mobil   │     │    11x │ mobil   │
│    2 │ mobil   │     │     5x │ minibus │
│    3 │ mobil   │     │     4x │ truk    │
│    4 │ mobil   │     └────────┴─────────┘
│    5 │ truk    │
│    6 │ mobil   │
│    7 │ mobil   │
│    8 │ minibus │
│    9 │ mobil   │
│   10 │ mobil   │
│   11 │ minibus │
│   12 │ mobil   │
└──────┴─────────┘
Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  2 │  2 │  3 │  3 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  5 │  5 │  5 │  6 │  6 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  5 │  5 │  5 │  9 │  9 │
├────┼────┼────┼────┼────┼────┼────┤
│  8 │  8 │  8 │ 10 │ 10 │ 12 │ 12 │
├────┼────┼────┼────┼────┼────┼────┤
│ 11 │ 11 │ 11 │  0 │  0 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

State Awal

Pada running kode ketiga dengan Iterasi/Generasi = 10, Populasi = 12, diperoleh informasi berikut:

Iterasi 0: maks = 310, rata-rata = 271.7
Iterasi 1: maks = 310, rata-rata = 278.1
Iterasi 2: maks = 310, rata-rata = 276.0
Iterasi 3: maks = 310, rata-rata = 277.5
Iterasi 4: maks = 310, rata-rata = 277.1
Iterasi 5: maks = 310, rata-rata = 274.4
Iterasi 6: maks = 310, rata-rata = 273.9
Iterasi 7: maks = 310, rata-rata = 273.0
Iterasi 8: maks = 310, rata-rata = 272.3
Iterasi 9: maks = 310, rata-rata = 271.0
Iterasi 10: maks = 310, rata-rata = 271.3

RINGKASAN EKSPERIMEN

Jumlah populasi awal : 12
Banyak iterasi : 10
Fitness akhir : 310
Durasi total : 0.02 detik (termasuk delay tampilan)
Durasi algoritma : 0.0154 detik (tanpa delay tampilan)

```
═══════════ POPULASI AWAL ═══════════

Menghasilkan populasi awal sebanyak 12 genetik...
Menghasilkan: [3, 2, 1, 3, 1, 0, 0, 0, 1, 3, 1, 1, 1, 0, 1, 1, 1]
Menghasilkan: [1, 1, 3, 1, 1, 0, 2, 0, 0, 1, 0, 1, 2, 3, 3, 0, 1]
Menghasilkan: [0, 2, 1, 1, 3, 1, 2, 0, 3, 0, 3, 1, 1, 2, 1, 1, 0]
Menghasilkan: [3, 1, 1, 0, 0, 0, 1, 2, 1, 0, 1, 1, 1, 2, 2, 3, 1]
Menghasilkan: [1, 1, 1, 3, 0, 1, 0, 0, 2, 1, 1, 1, 3, 3, 1, 2, 1]
Menghasilkan: [0, 1, 0, 3, 0, 0, 1, 1, 0, 1, 1, 2, 2, 0, 2, 2, 1]
Menghasilkan: [1, 2, 1, 3, 0, 2, 1, 1, 1, 2, 1, 1, 1, 1, 0, 0, 0]
Menghasilkan: [2, 2, 1, 0, 2, 1, 1, 3, 1, 0, 0, 1, 1, 1, 1, 1, 0]
Menghasilkan: [0, 3, 1, 2, 3, 2, 1, 1, 3, 1, 0, 2, 2, 0, 3, 2, 2]
Menghasilkan: [1, 1, 0, 1, 1, 1, 1, 1, 2, 1, 1, 1, 0, 1, 2, 1, 3]
Menghasilkan: [1, 1, 0, 1, 1, 2, 2, 1, 3, 2, 1, 0, 3, 1, 0, 1, 1]
Menghasilkan: [2, 1, 1, 1, 1, 1, 3, 1, 1, 1, 0, 1, 0, 1, 1, 2, 3]

═══════ KONDISI KAPAL AWAL ═══════

Mengecek kondisi awal kapal...
Kondisi kapal ukuran 7 x 5 dengan kapasitas maksimal 100 ton:
┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  0  │  0  │  0  │  0  │  0  │  0  │  0  │
└─────┴─────┴─────┴─────┴─────┴─────┴─────┘
```

State Akhir

```
═══════════ BEST GENETIC ═══════════

Fitness function tertinggi pada populasi akhir: 310
Diraih oleh individu dengan genetik: [2, 1, 1, 1, 1, 1, 3, 1, 1, 1, 0, 1, 0, 1, 1, 2, 3]
      PLACED                UNPLACED

┌──────┬─────────┐     ┌────────┬─────────┐
│ Code │ Type    │     │ Jumlah │ Jenis   │
├──────┼─────────┤     ├────────┼─────────┤
│    1 │ minibus │     │     9x │ mobil   │
│    2 │ mobil   │     │     6x │ minibus │
│    3 │ mobil   │     │     4x │ truk    │
│    4 │ mobil   │     └────────┴─────────┘
│    5 │ mobil   │
│    6 │ mobil   │
│    7 │ truk    │
│    8 │ mobil   │
│    9 │ mobil   │
│   10 │ mobil   │
│   11 │ mobil   │
│   12 │ mobil   │
│   13 │ mobil   │
└──────┴─────────┘

Matriks kapal setelah penempatan kendaraan:
┌────┬────┬────┬────┬────┬────┬────┐
│  1 │  1 │  1 │  2 │  2 │  3 │  3 │
├────┼────┼────┼────┼────┼────┼────┤
│  4 │  4 │  5 │  5 │  6 │  6 │  0 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  7 │  8 │  8 │  9 │  9 │
├────┼────┼────┼────┼────┼────┼────┤
│  7 │  7 │  7 │ 10 │ 10 │ 11 │ 11 │
├────┼────┼────┼────┼────┼────┼────┤
│ 12 │ 12 │ 13 │ 13 │  0 │  0 │  0 │
└────┴────┴────┴────┴────┴────┴────┘
```

2.3.2.3 Perbandingan Pengaruh Populasi dan Iterasi

Konfigurasi	Fitness akhir (3 run)	Rata-rata	Kenaikan maks, iterasi 0 sampai akhir (rata-rata)	Durasi algoritma (rata-rata)
Populasi 5, iterasi 5	310, 300, 320	310,0	+6,7	0,0066 s
Populasi 5, iterasi 10	310, 300, 310	306,7	+26,7	0,0165 s
Populasi 5, iterasi 15	320, 280, 320	306,7	+16,7	0,0149 s
Iterasi 10, populasi 3	310, 300, 310	306,7	+23,3	0,0102 s
Iterasi 10, populasi 6	300, 310, 330	313,3	+3,3	0,0110 s
Iterasi 10, populasi 12	320, 300, 310	310,0	+10,0	0,0159 s

Dari hasil analisis, ditemukan bahwa dari setiap iterasi, fitness number maksimum tidak pernah turun, hanya naik atau tetap. Hal ini disebabkan oleh populasi yang terus bertambah selalu menyebabkan individu yang terbaik tidak pernah terhapus. Adapun kenaikan dari fitness number tidak dijamin oleh iterasi, karena kenaikan terjadi dengan cenderung mendatar, dilihat dari iterasi 15x yang sempat memiliki fitness akhir lebih rendah daripada run dengan iterasi lebih sedikit, dan fakta bahwa iterasi 10x dan 15x memiliki rata rata yang lebih rendah daripada iterasi 5x. Hal ini bisa disebabkan karena populasi awal yang memang tidak beruntung, dan iterasi sebanyak apapun tetap sulit mengubah pengaruh dari populasi awal yang kurang bagus fitnessnya.

Pengaruh lain atas kurangnya pengaruh iterasi terhadap baiknya fitness adalah bahwa tiap iterasi hanya menambah 4 individu dan tidak ada yang dibuang, jadi anak dan hasil mutasi yang lebih buruk ikut menjadikan rata-rata lebih rendah. Pemilihan parent juga lemah karena fitness semua individu berdekatan (sekitar 250 sampai 320) sehingga peluang terpilihnya hampir sama.

Di sisi lain, populasi awal yang besar memberikan starting point yang lebih beragam, dilihat dari fitness akhir rata-rata 306,7 untuk populasi 3, 313,3 untuk populasi 6, dan 310,0 untuk populasi 12. Adapun populasi besar akan menyebabkan perubahan lebih lambat, hal ini karena setiap iterasi hanya menambah 4 individu. Di populasi 12, kontribusi anak dan mutan jadi proporsi kecil, jadi rata-rata populasi bergerak pelan.

Bab III: Kesimpulan dan Saran

3.1 Kesimpulan

Berikut kesimpulan hasil analisis berdasarkan pertanyaan analisis yang telah diberikan.

Pertanyaan	Jawaban berdasarkan Analisis
Seberapa dekat tiap-tiap algoritma bisa mendekati *global optima* dan mengapa hasilnya demikian?	Global optima teoritis adalah 340, dengan menggunakan hill climbing, value terdekat yang ditemukan adalah 320 dengan run ke-3 karena keterbatasan hill climbing. Dengan simulated annealing, berhasil ditemukan global optima 340 karena abilitas annealing untuk berpindah ke neighbor yang lebih buruk. Sementara itu, pada genetic algorithm, pencarian terdekat meraih 330 karena iterasi yang terbatas dan populasi awal yang mempengaruhi pencarian.
Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma *local search* yang lain?	Pada hasil pencarian Hill Climbing, karena skema yang digunakan adalah steepest ascent, value stuck pada nilai yang rendah dan tidak bisa mendapatkan nilai yang lebih tinggi daripada pencarian lain. Untuk pencarian Simulated Annealing, berhasil ditemukan global optima karena abilitas annealing untuk berpindah ke neighbor yang lebih buruk membuatnya lebih berpeluang untuk mendapatkan global optima. Sementara itu, pada genetic algorithm, karena iterasi yang terbatas dan populasi awal yang mempengaruhi pencarian, diperlukan iterasi yang lebih banyak untuk menemukan global optima.
Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?	Pada hill climbing, karena pencariannya yang sederhana dan cukup straightforward, waktu yang digunakan tidak selama algoritma lain. Pada Simulated Annealing, waktu yang dipakai untuk run cenderung lebih lama daripada algoritma lainnya, karena algoritma ini cenderung stuck dan perlu waktu yang lebih. Pada Genetic Algorithm, durasi algoritma meningkat seiring bertambahnya populasi dan iterasi, namun selisihnya hanya dalam milidetik, jadi Genetic Algorithm dapat memiliki pencarian yang lebih besar waktunya apabila di set menjadi iterasi yang cukup banyak.
Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?	Pada hasil hillclimbing, value akhir berkisar 270-320, hal ini karena initial state dan climb yang dilakukan cukup beragam. Pada hasil Simulated Annealing, hasil yang didapatkan cukup konsisten, yaitu 310-340, hal ini karena simulated annealing lebih berpeluang untuk mendapatkan global optima.

	Pada hasil Genetic Algorithm, fitness akhir berkisar 280-330, dan variasi antar run pada konfigurasi yang sama lebih besar daripada perbedaan antar konfigurasi.
Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?	Penambahan iterasi dan jumlah populasi cenderung menaikkan atau mempertahankan fitness maksimum, tetapi pengaruhnya kecil dan tidak konsisten pada 3 kali percobaan. Hal ini karena populasi awal yang dibangkitkan dengan acak dan nilai fitness yang diberikan sudah mendekati batas atasnya.

3.2 Saran

Berdasarkan hasil analisis, disarankan agar ketiga algoritma dikembangkan sesuai kelemahannya masing-masing. Hill Climbing dapat dilengkapi random restart atau sideways move agar tidak mudah terjebak di local optima, sedangkan Simulated Annealing dapat dioptimalkan melalui pengaturan suhu awal, laju pendinginan, dan kriteria berhenti untuk menekan durasi pencarian tanpa mengurangi kemampuannya mencapai global optima. Untuk Genetic Algorithm, jumlah iterasi dan populasi awal sebaiknya diperbesar karena selisih durasinya hanya dalam orde milidetik, disertai perbaikan mekanisme seleksi, seperti seleksi survivor dan tournament selection, serta peningkatan tingkat mutasi agar keragaman populasi terjaga. Selain itu, jumlah percobaan per konfigurasi sebaiknya ditambah dari tiga kali menjadi lebih banyak, dan durasi diukur dengan cara yang sama pada ketiga algoritma, sehingga perbandingan antaralgoritma dan antar parameter menjadi lebih meyakinkan.

Pembagian Tugas

Nama	NIM	Tugas
Endda Tsa Azzahra Syaifur	18224094	- Menyusun penyelesaian dengan algoritma Genetic Algorithm. - Menyusun laporan bagian Genetic Algorithm. - Membantu mengisi bagian kesimpulan dan saran. - Membantu mengisi beberapa bagian laporan seperti pembagian tugas dan form penggunaan AI. - Membantu merapikan beberapa bagian dokumen.
Rafael Sean Hadipranoto	18224100	- Menyusun penyelesaian dengan algoritma Simulated Annealing. - Menyusun laporan bagian Simulated Annealing. - Merapikan beberapa bagian dokumen. Mengisi bagian referensi laporan.
Gifta Mooira Prasharnissa	18224106	- Menyusun penyelesaian dengan algoritma Steepest Ascent Hill Climbing. - Menyusun laporan bagian Steepest Ascent Hill Climbing. - Membantu mengisi beberapa bagian laporan seperti pembagian tugas, form penggunaan AI, referensi, dan lampiran. - Membantu merapikan beberapa bagian dokumen.

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm

[2] leedonghyung28, "Knapsack Problem Algorithm — Python," Medium, 2024. [Online]. Available: https://medium.com/@dhleee0123/knapsack-problem-algorithm-python-cc19e191a78c

[3] Kie Codes, "Genetic Algorithm from Scratch in Python (tutorial with code)," YouTube, 2020. [Video]. Available: Genetic Algorithm from Scratch in Python (tutorial with code) .

[4] Slides Dosen, "Beyond Classical Search," slide kuliah IF3070 Dasar Inteligensi Artifisial, Institut Teknologi Bandung, 2026.

[5] kotarot, "rectangle-packing-solver: A Solver to Find a Solution of the 2D Rectangle Packing Problem by Simulated Annealing (SA) Optimization," GitHub. [Online]. Available: https://github.com/kotarot/rectangle-packing-solver

[6] "2D Knapsack Problem Using Simulated Annealing," ScienceDirect. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0305054810002510

[7] “Knapsack Problem Explained - Algorithms in Python," YouTube, 2024. [Video]. Available: https://www.youtube.com/watch?v=gCRw7BIrLPY

[8] "How to Optimally Pack a Container | 2D Knapsack," YouTube, 2024. [Video]. Available: https://www.youtube.com/watch?v=4CfV7A_RWrw

Lampiran

Repositori GitHub: https://github.com/giftamooi/IF3070_G06_unayahaprrri

Surat Pernyataan Penggunaan AI

Dengan ini, kami:

Nama/NIM : 1. Endda Tsa Azzahra Syaifur/18224094
2. Rafael Sean Hadipranoto/18224100
3. Gifta Mooira Prasharnissa/18224106
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelegensi Artifisial
Judul Tugas/Proposal : Tugas Besar I

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, jelaskan: Menggunakan VSCode inline suggestion	Ya	Sedang

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Endda Tsa Azzahra S.
18224094



Rafael Sean Hadipranoto
18224100



Gifta Mooira Prasharnissa
18224106